

CollabCoder: A Lower-barrier, Rigorous Workflow for Inductive Collaborative Qualitative Analysis with Large Language Models

Jie Gao

Singapore University of Technology
and Design, Singapore
Singapore-MIT Alliance for Research
and Technology, Singapore
gaojie056@gmail.com

Yuchen Guo

Singapore University of Technology
and Design, Singapore
yuchen_guo@mymail.sutd.edu.sg

Gionnieve Lim

Singapore University of Technology
and Design, Singapore
gionnievelim@gmail.com

Tianqin Zhang

Singapore University of Technology
and Design, Singapore
tianqin_zhang@mymail.sutd.edu.sg

Zheng Zhang

University of Notre Dame, USA
zzhang37@nd.edu

Toby Jia-Jun Li

University of Notre Dame, USA
toby.j.li@nd.edu

Simon Tangi Perrault

Singapore University of Technology
and Design, Singapore
perrault.simon@gmail.com

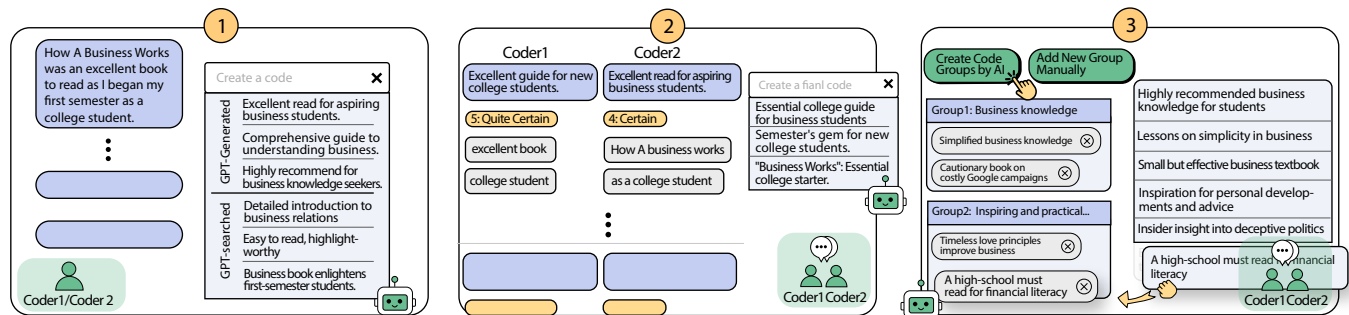


Figure 1: CollabCoder, A Lower-barrier, Rigorous Workflow for Inductive Collaborative Qualitative Analysis. The workflow consists of three key stages: 1) Independent Open Coding, facilitated by on-demand code suggestions from LLMs, producing initial codes; 2) Iterative Discussion, focusing on conflict mediation within the coding team, producing a list of agreed-upon code decisions; 3) Codebook Development, where code groups may be formed through LLM-generated suggestions, based on the list of decided codes in the previous phase.

ABSTRACT

Collaborative Qualitative Analysis (CQA) can enhance qualitative analysis rigor and depth by incorporating varied viewpoints. Nevertheless, ensuring a rigorous CQA procedure itself can be both complex and costly. To lower this bar, we take a theoretical perspective to design a one-stop, end-to-end workflow, CollabCoder, that integrates Large Language Models (LLMs) into key *inductive*

CQA stages. In the independent open coding phase, CollabCoder offers AI-generated code suggestions and records decision-making data. During the iterative discussion phase, it promotes mutual understanding by sharing this data within the coding team and using quantitative metrics to identify coding (dis)agreements, aiding in consensus-building. In the codebook development phase, CollabCoder provides primary code group suggestions, lightening the workload of developing a codebook from scratch. A 16-user evaluation confirmed the effectiveness of CollabCoder, demonstrating its advantages over the existing CQA platform. All related materials of CollabCoder, including code and further extensions, will be included in: <https://gaojie058.github.io/CollabCoder/>.

CCS CONCEPTS

• **Human-centered computing** → Collaborative and social computing systems and tools.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '24, May 11–16, 2024, Honolulu, HI, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0330-0/24/05...\$15.00

<https://doi.org/10.1145/3613904.3642002>

KEYWORDS

Collaborative Qualitative Analysis, Large Language Models, Grounded Theory, Inductive Qualitative Coding

ACM Reference Format:

Jie Gao, Yuchen Guo, Gionnieve Lim, Tianqin Zhang, Zheng Zhang, Toby Jia-Jun Li, and Simon Tangi Perrault. 2024. CollabCoder: A Lower-barrier, Rigorous Workflow for Inductive Collaborative Qualitative Analysis with Large Language Models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 29 pages. <https://doi.org/10.1145/3613904.3642002>

1 INTRODUCTION

Rigor and in-depth interpretation are primary objectives in qualitative analysis [46, 66]. Collaborative Qualitative Analysis (CQA) can achieve these objectives by mandating researchers to code individually and then converge on interpretations through iterative discussions [2, 17, 33, 50, 60] (see Figure 2).

However, adhering strictly to the CQA's prescribed workflow, which is necessary for achieving both rigor and depth goals, poses challenges due to its *inherent complexity* and *time and labor costs*. For the former issue, the complex requirements of the CQA process, which involves multiple steps with specific requirements for each, presents a considerable entry barrier for those less experienced or unfamiliar with CQA standards like graduate students, early-career researchers, and diverse research teams, etc [17, 60]. For instance, while open coding necessitates coders to work independently, subsequent processes demand collaboration. Often, coders need to toggle between coding on their own and collaborating to refine the codes. However, the CQA software Atlas.ti Web lacks an independent coding space [26]. This means the coding process is always visible to everyone, potentially influencing other coders' choices. Similarly, for the latter issue, the iterative nature of CQA requires the involvement and coordination of many coders [25, 26]. However, the conventional CQA tools such as MaxQDA, NVivo, and Google Docs/Sheets are not specifically designed for this aspect but for basic functions, such as proposing codes, which may only offer trivial assistance in qualitative analysis. They necessitate additional team coordination steps [22, 47], like document downloading, data sharing, data importing, manual searching, and crafting codebook tables. **This inconsistency between theories and practical software can lead to confusion and potentially result in incorrect or suboptimal practices.** For instance, coders might opt for individual coding to gain efficiency, which leads to fewer interactive discussions and, ultimately, outcomes that reflect the individual coder's inherent biases [2, 17].

In academics, current HCI research mainly focuses on addressing the effort-intensive challenge of CQA process. For example, Zade et al. [68] suggested enabling coders to order different states of disagreements by conceptualizing disagreements in terms of tree-based ranking metrics of diversity and divergence. Aeonium [18] allows coders to highlight ambiguity and inconsistency and offers features to navigate through and resolve them. With the growing prevalence of AI, Gao et al. [26] underscores the potential of AI in CQA through CoAICoder. They suggest that AI agents providing code suggestions based on teams' coding histories could accelerate

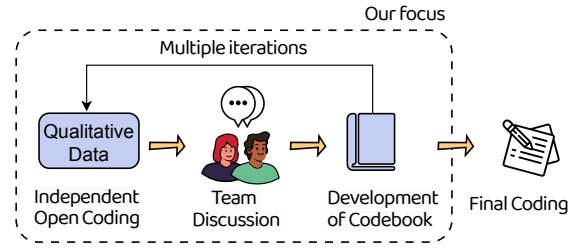


Figure 2: Collaborative Qualitative Analysis (CQA) [15, 16, 60] is an iterative process involving multiple rounds of iteration among coders to reach a final consensus. Our goal with CollabCoder is to assist users across key stages of the CQA process.

collaborative efficiency and foster a shared understanding more quickly at the early stage of coding.

With the advancements of Large Language Models (LLMs)¹ like GPT-3.5 and GPT-4², they have been pivotal in enhancing qualitative analysis due to the exceptional abilities in understanding and generating text. Atlas.ti Web, a commercial platform for qualitative analysis, integrated OpenAI's GPT model on March 28, 2023³. This integration offers functionalities like one-click code generation and AI-driven code suggestions, significantly streamlining the coding process. Moreover, LLMs are being explored for their assistance in deductive coding [67] and for achieving outcomes comparable to human-level qualitative interpretations [8].

While this existing research provides valuable insights into various facets of CQA, there has been little emphasis on creating a streamlined workflow to bolster the rigorous CQA process. Building upon well-accepted CQA steps that are deeply rooted in Grounded Theory [13] and Thematic Analysis [45], we aim to address this gap by presenting a holistic solution that streamlines the CQA process, with an emphasis on the *inductive* qualitative analysis, central to the development of the codebook and coding schema. **Our primary objective is to lower the bar of adherence to the rigorous CQA process**, thereby providing a potential for enhancing the quality of qualitative interpretation [14] with controllable and manageable effort.

To this end, we introduce CollabCoder, a CQA workflow within a web application system that integrates LLMs for the development of code schemes. Primarily, CollabCoder features interfaces tailored to a three-stage CQA workflow, aligned with the standard CQA process. It facilitates real-time data synchronization and centralized management, obviating the need for intricate data exchanges among coders. The platform offers both individual and shared workspaces, facilitating seamless transitions between personal and

¹In this paper, AI and LLMs are used interchangeably to refer to the broader field of Artificial Intelligence, specifically large language models. GPT, as an example of a large language model, specifically refers to products developed by OpenAI, such as ChatGPT.

²<https://openai.com/blog/introducing-chatgpt-and-whisper-apis>

³<https://atlasti.com/ai-coding-powered-by-openai>

collaborative settings at various stages. The shared workspace contains collective decision-making data and quantitative metrics, essential for addressing code discrepancies. Beyond basic functionalities, CollabCoder integrates GPT to achieve multiple goals: 1) providing automated code suggestions to streamline open codes development; 2) aiding the conversion of open codes into final code decisions; and 3) providing initial versions of code groups, derived from these code decisions, for coders to further refine and adjust.

We conducted an evaluation of the CollabCoder system, addressing the following research questions:

- **RQ1.** Can CollabCoder support qualitative coders to conduct CQA effectively?
- **RQ2.** How does CollabCoder compare to currently available CQA tools like Atlas.ti Web?
- **RQ3.** How can the design of the CollabCoder be improved?

Our evaluation of CollabCoder demonstrated its user-friendliness, particularly for beginners navigating the CQA workflow (75%+ participants agree or strongly agree on "*easy to use*" and "*learn to use quickly*"). It effectively supports coding independence, fosters an understanding of (dis)agreements, and helps in building a shared understanding within teams (75%+ participants agree or strongly agree that CollabCoder helps to "*identify (dis)agreements*", "*understand others' thoughts*", "*resolve disagreements*" and "*understand the current level of agreement*"). Additionally, CollabCoder optimizes the discussion phase by allowing code pairs to be resolved in a single dialogue, in contrast to Atlas.ti Web where only a few codes are typically discussed. This minimizes the need for multiple discussion rounds, thereby boosting collaborative efficiency. Regarding the role GPT plays in the CQA workflow, we emphasize the need to balance LLM capabilities with user autonomy, especially when GPT acts as a "*suggestion provider*" during the initial phase. In the discussion stages, GPT functions as a "*mediator*" and "*facilitator*", aiding in efficient and equitable team decision-making as well as in the formation of code groups.

Our work with CollabCoder consequently paves the way for LLMs-powered (C)QA tools and also uncovers critical challenges and insights for both human-AI and human-human interactions within the context of qualitative analysis. We make the following contributions:

- (1) Outlining design guidelines, informed by both theories and practices, that shaped the development of CollabCoder and may inspire future AI-assisted CQA system designs.
- (2) Developing CollabCoder that incorporates LLMs into different steps of the inductive CQA workflow, enabling coders to seamlessly transition from independent open coding to the aggregation of code groups.
- (3) Conducting an evaluation of CollabCoder, yielding valuable insights from user feedback on both the coding and collaboration experiences, which also shed light on the role of LLMs throughout different stages of the CQA process.

2 BACKGROUND OF QUALITATIVE ANALYSIS

Qualitative analysis is an important methodology in HCI and social science for interpreting data from interviews, focus groups, observations, and more [24, 43]. The goal of qualitative analysis is to transform unstructured data into detailed insights regarding key

aspects of a given situation or phenomenon, addressing researchers' concerns [43]. Commonly employed strategies include Grounded Theory (GT) [24] and Thematic Analysis (TA) [45].

GT is originally formulated by Glaser and Strauss [24, 29]. Its primary objective is to abstract *theoretical conceptions* based on *descriptive* data [7, 15]. A primary approach in GT involves *coding*, specifically assigning *codes* to data segments. These conceptual *codes* act as crucial bridges between descriptive data and theoretical constructs [7]. In particular, GT coding involves two key phases: initial and focused coding. In initial coding, researchers scrutinize data fragments—words, lines, or incidents—and add codes to them. During focused coding, researchers refine initial codes by testing them against a larger dataset. Throughout, they continuously compare data with both other data and existing codes [10], to build theoretical conceptions or theories. Similarly, TA is another method commonly used for analyzing qualitative data, aimed at identifying, analyzing, and elucidating recurring themes within the dataset [6, 45].

Several practical frameworks exist for conducting CQA [7, 33, 60]. Particularly, Richards et al. [60] have proposed a six-step methodology rooted in GT and TA. The methodology encompasses the following steps: ①preliminary organization and planning: An initial team meeting outlines project logistics and sets the overall analysis plan; ②open and axial coding: Team members use open coding to identify concepts and patterns, followed by axial coding to link these patterns [15, 24]; ③development of a preliminary codebook: One team member reviews the codes and formulates an initial codebook; ④pilot testing the codebook: Researchers independently code 2-3 transcripts and record issues with the initial codebook; ⑤final coding process: The updated codebook is applied to all data, including initially-coded transcripts; and ⑥review and finalization of the codebook and themes: After coding all the transcripts, the team holds a final meeting to finalize the codebook.

Richards et al. also delineate two distinct CQA approaches: *consensus coding* and *split coding*. *Consensus coding* is more rigorous but time-consuming; each coder independently codes the same data and then engages in a team discussion to resolve disagreements and reach a consensus. Conversely, *split coding* is quicker but less rigorous, with coders working on separate data sets. This method leans heavily on the clarity established during the preliminary coding phases and pre-defined coding conventions.

Drawing on the six-step CQA framework by Richards et al., **CollabCoder is designed to streamline crucial stages of the CQA workflow**. It places particular emphasis on the **consensus coding** approach, ensuring thorough data discussions and complete resolution of disagreements. It also focus on **inductive qualitative analysis**, wherein both codes and the codebook evolve during the analytical process. This is in contrast to the work by Xiao et al. [67], which prioritizes the use of LLMs to assist deductive coding based on a pre-existing codebook. The overarching aim is to lower the bar for maintaining the rigor and depth of the inductive CQA process.

In the following, we describe the terms and concepts used in this paper:

- **Code:** A code is typically a succinct word or phrase created by the researcher to encapsulate and interpret a segment

of qualitative data. This facilitates subsequent pattern detection, categorization, and theory building for analytical purposes [62].

- **Coding:** Coding serves as a key method for analyzing qualitative data. It involves labeling segments of data with codes that concurrently categorize, encapsulate, and interpret each individual data point [24].
- **Codebook/Themes/Code groups:** A codebook is a hierarchical collection of code categories or thematic structure, typically featuring first and second-order themes or code groups, definitions, transcript quotations, and criteria for including or excluding quotations [60, 62].
- **Agreement/Consensus:** Agreement or consensus is attained through in-depth discussions among researchers, where divergent viewpoints are scrutinized and potentially reconciled following separate rounds of dialogue [50]. The degree of agreement among multiple coders serves as an indicator of the analytical rigor of a study [17].
- **Inter-coder Reliability (IRR):** IRR is a numerical metric aimed at quantifying agreement among multiple researchers involved in coding qualitative data [50, 57].
- **Coding Independence:** Typically, open coding and initial code development are undertaken independently by individual team members to minimize external influence on their initial coding choices [17, 33].
- **Data units/Unit-of-analysis:** The unit-of-analysis (UoA) specifies the granularity at which codes are made, such as at the flexible or sentence level [61].

3 RELATED WORK

3.1 Existing Tools for CQA

Researchers have proposed multiple platforms and approaches to facilitate CQA [18, 25, 28]. For instance, Aeonium [18] assists coders by flagging of uncertain data, highlighting discrepancies, and permitting additional code definitions and coding history checks. Code Wizard [25], an Excel-embedded visualization tool, supports the code merging and analysis process of CQA by allowing coders to aggregate each individual coding table, automatically sort and compare the coded data, calculate IRR, and generate visualization results. Zade et al. [68] suggest sorting the text according to its ambiguity, allowing coders to concentrate on disagreements and ambiguities to save time and effort. The primary goal of these works is to simplify code comparison and identify disagreements and ambiguities [12], thereby enhancing code understanding among coders and streamlining the consensus-building process.

There are also several commercial software (e.g., NVivo⁴, MaxQDA⁵, and Atlas.ti Web⁶) available that support collaborative coding in various ways. For code comparison and discussion, these systems enable users to export and import individually coded documents, facilitating line-by-line, detailed discussions among the coding team for conflict resolution and code consolidation. They also permit coders to add memos for recording concerns and ambiguities to be addressed in discussions. Specifically, Atlas.ti Web (very different

from its local version) allows coders to collaborate in real-time within a shared online space for data and code sharing, a feature also present in Google Docs, albeit not tailored for CQA. While it aligns closely with our objective of streamlining CQA workflows by eliminating the need for downloading or uploading documents, they lean towards a "less rigorous" coding method. Given that codes and data are persistently visible to all coders, they do not facilitate "independent" open coding within a coding team. This feature is also available in the latest version of NVivo's collaboration tools⁷.

Our work on CollabCoder enriches the existing literature by offering a one-stop, end-to-end workflow that seamlessly transitions the output of one stage into the input for the next. It also leverages prior design considerations to aid coders in reaching consensus. Ultimately, our objective with CollabCoder is to streamline the entire CQA process, thereby lowering the barriers to adopting a rigorous approach. This rigor manifests through the inclusion of key CQA stages, the preservation of coding independence, and the fostering of thorough discussions that lead to informed coding decisions based on both agreements and disagreements.

3.2 AI-assisted (C)QA Systems

While the utilization of AI to aid in different aspects of the qualitative coding process has garnered increasing interest [12, 23, 40, 53], the majority of current research focuses on leveraging AI to aid individual qualitative coding.

Feuston et al. [23] outlined various ways AI can be beneficial at different stages of QA. For example, AI can provide preliminary insights into large data sets through semantic network analysis before formal inductive coding begins. It can also suggest new codes based on the initial coding work already performed by coders. Particularly relevant to our research with CollabCoder is their emphasis on coding stages: "when, how, and whether" to introduce AI is an important consideration. This aligns with our exploration that AI should, and can, perform different functions and have distinct task allocations at various key stages of CQA.

For system work, Cody [61] utilizes supervised techniques to enable researchers to define and refine code rules that extend coding to unseen data, while PaTAT [27] provides a program synthesizer that learns human coding patterns and serves as a reference for users. Scholastic [34] partially shares our goal, specifically aiming to maintain a focused workflow while utilizing codes generated by coders as input for subsequent stages, such as a learning pattern for AI, and as filters to visualize the distribution of emerging knowledge clusters. On the collaboration side, Gao et al. [26] have identified opportunities to use AI to facilitate CQA efficiency at the early stage of CQA. They contend that utilizing a shared AI model, trained on the coding team's past coding history, can expedite the formation of a shared understanding among coders.

Although these AI-related works utilize traditional AI technologies rather than the latest LLMs, their insights and design considerations have informed the development of our CollabCoder workflow. They prompt us to further consider AI's role throughout the process, as well as the potential advantages and concerns that such assistance might introduce.

⁴<https://lumivero.com/products/collaboration-cloud/>

⁵<https://www.maxqda.com/help-mx20/teamwork/can-maxqda-support-teamwork>

⁶<https://atlasti.com/atlas-ti-web>

⁷Accessed on September 13, 2023.

3.3 Using LLMs in Qualitative Analysis

Recent advancements in LLMs like GPT-3.5 and GPT-4 offer promising text generation, comprehension, and summarization capabilities [56]. To enhance coding efficiency, Atlas.ti Web has incorporated OpenAI's GPT models to provide one-click code generation and AI-assisted code suggestions⁸. Other software predominantly depend on manual human evaluation or basic AI applications, such as word frequency counting or sentiment analysis.

On the research side, Byun et al. [8] posed the question: "Can a model possess experiences and utilize them to interpret data?" They examined various prompts to assess theme generation by models such as *text-davinci-003*, a fine-tuned variant, and ChatGPT (referred to as gpt-turbo in their experiment). Their approach involved methods like one-shot prompting and question-only techniques. Their findings suggested that these models are adept at producing human-like themes and posing thought-provoking questions. Furthermore, they discovered that subtle changes in the prompt — transitioning from "important" to "important HCI-related" or "interesting HCI-related" — yielded more nuanced results. Additionally, Xiao et al. [67] demonstrated the viability of employing GPT-3 in conjunction with an expert-developed codebook for deductive coding. Their findings showcased a notable alignment with expert ratings on certain dimensions. Moreover, the codebook-centered approach surpassed the performance of the example-centered design. They also mentioned that transitioning from a zero-shot to a one-shot scenario profoundly altered the performance metrics of LLMs.

In summary, while research on CQA has explored code comparison and the identification of disagreements in specific phases, as well as the use of AI and LLMs for (semi-)automated qualitative analysis, a comprehensive end-to-end workflow that lowers the barrier for user adherence to the standard CQA workflow remains largely unexplored. Furthermore, the seamless integration of LLMs into this workflow, along with the accompanying benefits and concerns, remains an unexplored avenue. Our proposed workflow, CollabCoder, aims to cover key stages such as independent open coding, iterative discussions, and codebook development. Additionally, we offer insights into the integration of LLMs within this workflow.

4 DESIGN GOALS

We aim to create a workflow that aligns with standard CQA protocols with a lower adherence barrier, while also integrating LLMs to boost efficiency.

4.1 Method

To achieve our goals, we extracted 8 design goals (DG) for CollabCoder from three primary sources (see Table 1).

Step1: Semi-systematic literature review. We initially reviewed established theories and guidelines on qualitative analysis. Given our precise focus on theories such as Grounded Theory and Thematic Analysis and our emphasis on their particular steps, we used a semi-systematic literature review method [48, 63]. This method is particularly aimed at identifying key themes relevant to a specific topic while offering an appropriate balance of depth and flexibility. Our results are incorporated into the background section, as

detailed in Section 2, aiming to establish a robust theoretical foundation for our work. It also assists in understanding the inputs, outputs, and practical considerations for each stage of CollabCoder workflow. This method formulates DG1, DG2, DG4, DG5, DG6, DG7.

Step2: Examination of prevalent CQA platforms. The semi-systematic literature review was followed by triangulation with existing qualitative analysis platforms, for which we assessed the current state of design by examining their public documents and official websites (the detailed examination are summarized in Appendix Table 4). This examination enables us to gain insights into the critical features, advantages, and drawbacks of these CQA platforms, such as the dropdown list for the selection of historical codes and the calculation of essential analysis metrics. As a result of this triangulation, we successfully extracted new design goals, DG3 and DG8, and refined the existing DG1 and DG2.

Step3: Preliminary interviews with researchers with qualitative analysis experience. Based on the primary understanding of the CQA theories, and the primary version of 8 DGs, we subsequently developed the primary prototype (see Appendix Figures 9, 10, and 11). We utilized the initial version of CollabCoder to conduct a pilot interview evaluation with five researchers possessing at least one year of experience in qualitative analysis (refer to Table 2). The aim was to gather expert insights into the workflow, features, and design scope of the theory-driven CollabCoder, thereby refining our design goals and adjusting the prototype's primary features. During the evaluation, the researchers were first introduced to the CollabCoder prototype. Subsequently, they shared their impressions, raised questions, and offered suggestions for enhancements. We transcribed their interview audio and did a thematic analysis on the interview transcriptions (see analysis results in Appendix Figure 12) and refined two of the design goals (DG1 and DG4) based on their feedback.

4.2 Results for Design Goals

DG1: Supporting key CQA phases to encourage stricter adherence to standardized CQA processes. Our primary goal is the creation of a mutually agreed codebook among coders, essentially focusing on the inductive qualitative analysis process. Therefore, from the six-step CQA methodology [60], we are particularly concerned with "open and axial coding", "iterative discussion", and the "development of a codebook".

Although complying with CQA steps is critical for deriving robust and trustworthy data interpretations [60], the existing software workflows and AI integrations significantly diverge from theoretical frameworks. These systems lack a centralized and focused workflow; there is a noticeable absence of fluidity between stages, where the output of one phase should ideally transition seamlessly into the input of the next. This deficiency complicates the sensemaking process among coders and often discourages them from adhering to the standardized CQA workflow. This sentiment is mirrored by an expert (P1) who remarked, "In a realistic scenario, how many people do follow this [standard] flow? I don't think most people follow."

In response, we have tailored a workflow that integrates the key CQA stages we identified. This streamlined process assists

⁸<https://atlasti.com/atlas-ti-ai-lab-accelerating-innovation-for-data-analysis>

Table 1: Summary of Sources Informing Our Design Goals

Sources	Content	Design Goals (DG)
Step1: Semi-systematic literature review	insights into the key phases of CQA theories, including grounded theory and thematic analysis, detailing inputs, outputs, and practical considerations for each.	DG1, DG2, DG4, DG5, DG6, DG7
Step2: Examination of prevalent CQA platforms	insights into essential features, pros, and cons of key CQA platforms, such as Atlas.ti Web ⁹ , MaxQDA Team Cloud ¹⁰ , NVivo Collaboration Cloud ¹¹ , and Google Docs.	DG1, DG2, DG3, DG8
Step3: Preliminary interviews with researchers with qualitative analysis experience	insights into CollabCoder workflow, features, and design scope through expert feedback, concerns, and recommendations.	DG1, DG4

Table 2: Participant Demographics in Exploration Interview

No.	Fields of Study	Current Position	QA Software	Years of QA
P1	HCI, Ubicomp	Postdoc Researcher	Atlas.ti	4.5
P2	HCI, NLP	PhD student	Google Sheet/Whiteboard	4
P3	HCI, Health	PhD student	Google Sheet	4
P4	HCI, NLP	PhD student	Excel	1.5
P5	Software Engineering	PhD student	Google Sheet	1

the coding team in aligning with the standard coding procedure, ensuring results from one phase transition seamlessly into the next. Our goal is to simplify adherence to the standard workflow by making it more accessible.

DG2: Supporting varying levels of coding independence at each CQA stage to ensure a strict workflow. In Grounded Theory [15, 60], a primary principle is to enable coders to independently produce codes, cultivate insights from their own viewpoints, and subsequently share these perspectives at later stages. The switch between independent coding and collaborative discussion can always occur during multiple iterations. However, we have found that widely-used platforms such as Atlas.ti Web and NVivo, while boasting real-time collaborative coding features, fall short in providing robust support for independent coding. The persistent visibility of all raw data, codes, and quotations to all participants may potentially bias the coding process. Moreover, Gao et al. [26] found that in scenarios prioritizing efficiency, some coders are willing to compromise independence, which could potentially impact coding rigor.

In response, our workflow designates varying levels of coder independence at different stages: strict separation during the independent open coding phase and mutual code access in the discussion and code grouping phases. We aim to ensure that coders propose codes from their unique perspectives, rather than prematurely integrating others' viewpoints, which could compromise the final coding quality.

DG3: Supporting streamlined data management and synchronization within the coding team. While Atlas.ti Web has faced criticism for its lack of support for coder independence [26], as outlined in DG2, it does offer features like **synchronization** and

centralized data management. Through a web-based application, these features allow teams to manage data preprocessing and share projects, ensuring seamless coding synchronization among members. The sole requirement for participation is a web browser and an Atlas.ti Web account. In contrast, traditional software like MaxQDA and NVivo lack these capabilities. This absence necessitates additional steps, such as locally exporting coding documents post-independent coding and then sharing them with team members¹². These steps may introduce obstacles to a smooth and focused CQA process. However, as mentioned in DG2, Atlas.ti Web sacrifices coding independence.

In response, we strive to strike a balance between data management convenience and coding independence, facilitating seamless data synchronization and management via a web application while maintaining design features that support independent coding.

DG4: Supporting interpretation at the same level among coders for efficient discussion. As per Saldana's qualitative coding manual [62], coders may use a "splitter" (e.g., line-by-line) or a "lumper" (e.g., paragraph-by-paragraph) approach. This variation can lead coders to work on different levels of granularity, resulting in many extra efforts to align coding units among coders for line-by-line or code-by-code comparison, in order to make them on the same level to determine if they have an agreement or consensus, not to mention the calculation of IRR [26]. Therefore, standardizing and aligning data units for coding among teams is essential to facilitate efficient code comparisons and IRR calculations [25, 42, 57]. Two prevalent approaches to achieve this are: 1) allowing the initial coder to finish coding before another coder commences work on the same unit [20, 42, 57], and 2) predefining a fixed text unit for

¹²<https://www.maxqda.com/help-mx20/teamwork/can-maxqda-support-teamwork>

the team, such as sentences, paragraphs, or conceptually significant "chunks" [42, 57].

In response, we aim to enhance code comparison efficiency by offering coders predefined coding unit options on CollabCoder, thereby ensuring alignment between their interpretations. However, it is important to recognize an intrinsic trade-off between **unit selection flexibility** and **effort expenditure**. While reduced flexibility can decrease the effort needed to synchronize coders' understanding in discussions, it may also constrain users' freedom in coding. According to expert feedback, our workflow represents an "ideal" scenario. As one expert (P3) noted, *"I think overall the CollabCoder workflow pretty interesting... However, I think the current workflow is a very perfect scenario. What you haven't considered is that in qualitative coding, there's often a sentence or a section that can be assigned to multiple codes. In your current case, you are assigning an entire section into just one code."* Additionally, our proposed workflow appears to operate under the assumption that coding is applied to specific, isolated units, failing to account for instances where the same meaning is distributed across different data segments. *"Because sometimes [for a code] you need one part of one paragraph, the other part is in another paragraph. right?"* (P1)

DG5: Supporting coding assistance with LLMs while preserving user autonomy. As Jiang et al. [40] suggested, AI should not replace human autonomy, participants in their interview said that *"I don't want AI to pick the good quotes for me..."*. AI should only offer recommendations when requested by the user, after they have manually labeled some codes, and support the identification of overlooked codes based on existing ones. To control user autonomy, the commercial software, Atlas.ti Web, has transitioned from auto-highlighting quotations and generating code suggestions via LLMs for all documents with a single click, to now allowing users to request such suggestions on demand¹³. The platform's earlier AI-driven coding, although time-saving, compromised user control in the coding process.

In response, we emphasize user autonomy during the coding process, letting coders first formulate their own codes and turning to LLM assistance only upon request.

DG6: Facilitating deeper and higher-quality discussion. As highlighted in Section 2, CollabCoder's primary objective is to foster consensus among coders [2, 60]. This demands quality discussions rooted in common ground [55, 58] or shared mental model [30, 31]. Common ground pertains to the information that individuals have in common and are aware that others possess [4, 13]. This grounding is achieved when collaborators engage in deep communication [4]. Similarly, shared mental model is a conception in team coordination theory [22, 47]. The development of this shared mental model can enable team members to anticipate one another's needs and synchronize team members' efforts, facilitating implicit coordination without the necessity for explicit interaction [22]. This becomes particularly valuable in enabling high-quality and efficient coordination, especially when time is limited [38].

In response, we aim to establish common ground or shared mental model among the team to 1) facilitate deeper and higher-quality

discussion by surfacing underlying coding disagreements; 2) concentrate coders' efforts on the most critical parts that need the most discussion [18, 68].

DG7: Facilitating cost-effective, fair coding outcomes and engagement via LLMs. Once the common ground is established, achieving a coding outcome that is cost-effective, fair, and free from negative effects becomes a challenging yet crucial task [21, 39]. To reach a consensus, the team often engages in debates or invests time crafting code expressions that satisfy all coders [21], significantly prolonging the discussion. In addition, Jiang et al. [40] reveal that team leaders or senior members may have the final say on the codes, potentially introducing bias.

In response, our objective is to foster deep, efficient, and balanced discussions within the coding team. We ensure that every coder's prior open coding decisions are respected, allowing them to actively participate in both discussions and the final decision-making process, with the support of LLMs.

DG8: Enhancing the team's efficiency in code group generation. Prevalent QA software like Atlas.ti, MaxQDA, and NVivo prominently feature a code manager. This tool lets coders track, modify, and get a holistic view of their current code assignments. It plays a vital role in facilitating discussions, proposing multiple code groups, and aiding code reuse during coding. Meanwhile, Feuston et al. [23] noted some participants used AI tools to auto-generate final code groups from human-assigned codes.

In response, we offer the code manager that allows for manual editing and adjustment of code groups. Additionally, we aim to integrate automatic code group generation to streamline the coding process via the assistance of LLMs.

5 COLLABCODER SYSTEM

With the aforementioned design goals in mind, we finalized the CollabCoder system and its CQA workflow (refer to Figure 3).

5.1 CollabCoder Workflow & Usage Scenario

We introduce an example scenario to demonstrate the usage of CollabCoder (see Figure 5). Suppose two coders **Alice** and **Bob** are conducting qualitative coding for their data. The lead coder, **Alice**, first creates a new project on CollabCoder, then imports the data, specifies the level of coding as "paragraph", and invites **Bob** to join the project. After clicking on CREATE PROJECT, CollabCoder's parser will split the imported raw data into units (paragraph in this case). The project can then be shown on both coders' interfaces.

5.1.1 Phase 1: Independent Open Coding. In Phase 1, **Alice** and **Bob** individually formulate codes for each unit in their separate workspaces via the same interface. Their work is done independently, with no visibility into each other's codes. If **Alice** wants to propose a code for a sentence describing a business book for students, she can either *craft* her own code, choose from code recommendations generated by the GPT model (e.g., "Excellent guide for new college students", "Insightful read on business fundamentals", "How A Business Works": semester's gem), or *pick* one of the top three most relevant codes discovered by GPT in her coding history, and making modifications as needed. She can then *select* relevant keywords/phrases (e.g., "excellent book",

¹³<https://atlasti.com/atlas-ti-ai-lab-accelerating-innovation-for-data-analysis>, accessed on 14th August 2023

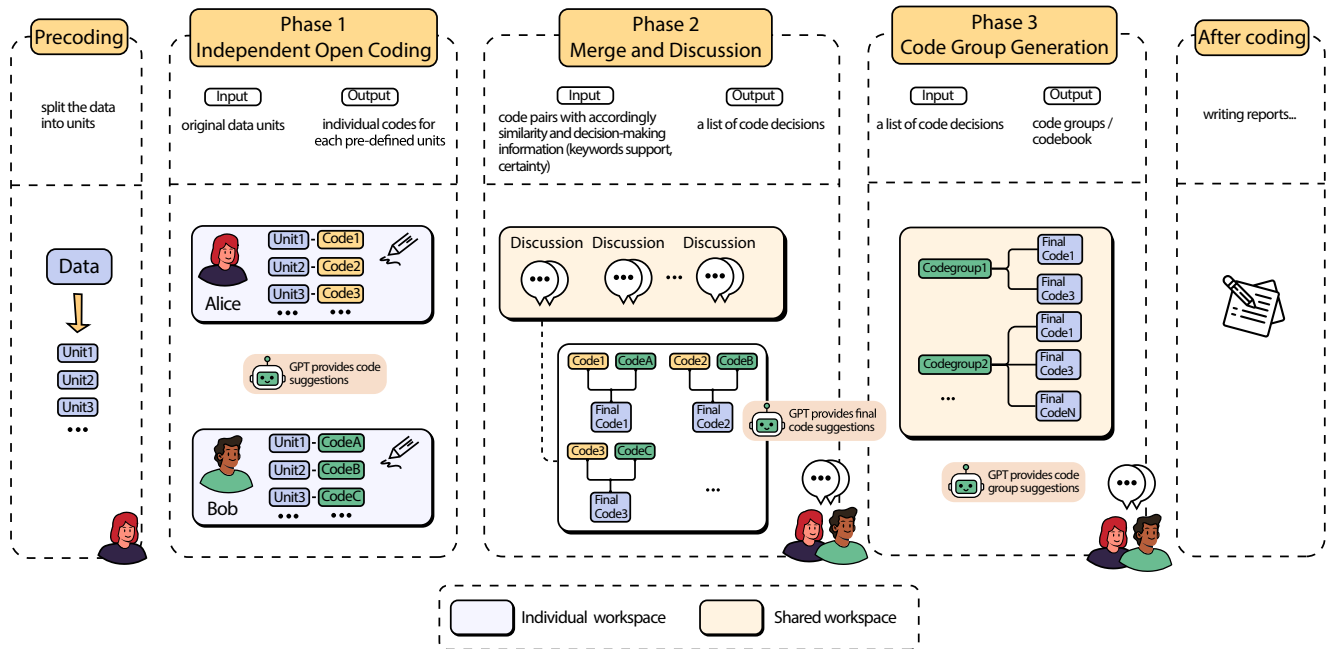


Figure 3: CollabCoder Workflow. The lead coder Alice first splits qualitative data into small units of analysis, e.g., sentence, paragraph, prior to the formal coding. Alice and Bob then: Phase 1: independently perform open coding with GPT assistance; Phase 2: merge, discuss, and make decisions on codes, assisted by GPT; Phase 3: utilize GPT to generate code groups for decided codes and perform editing. They can write reports based on the codebook and the identified themes after the formal coding process.

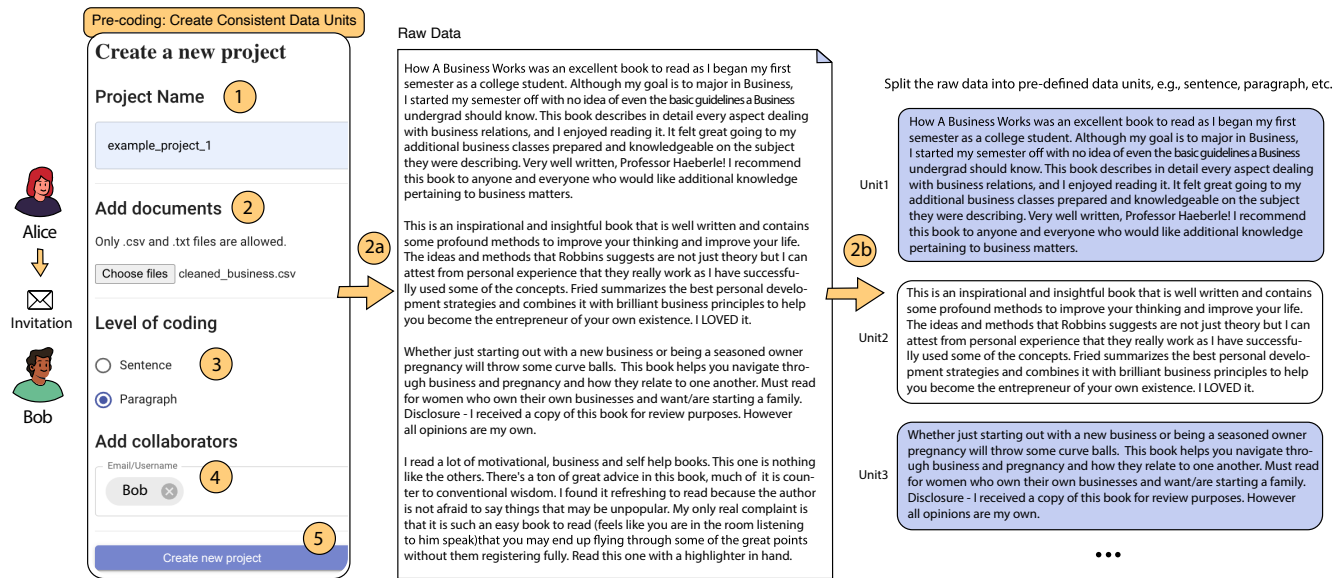


Figure 4: Pre-coding: establish consistent data units and enlist coding team during project creation. The primary coder, Alice, can: 1) name the project, 2) incorporate data, ensuring it aligns with mutually agreed data units, 2a) illustrate how CollabCoder manages the imported data units, 3) define the coding granularity (e.g., sentence or paragraph), 4) invite a secondary coder, Bob, to the project, and 5) initiate the project.

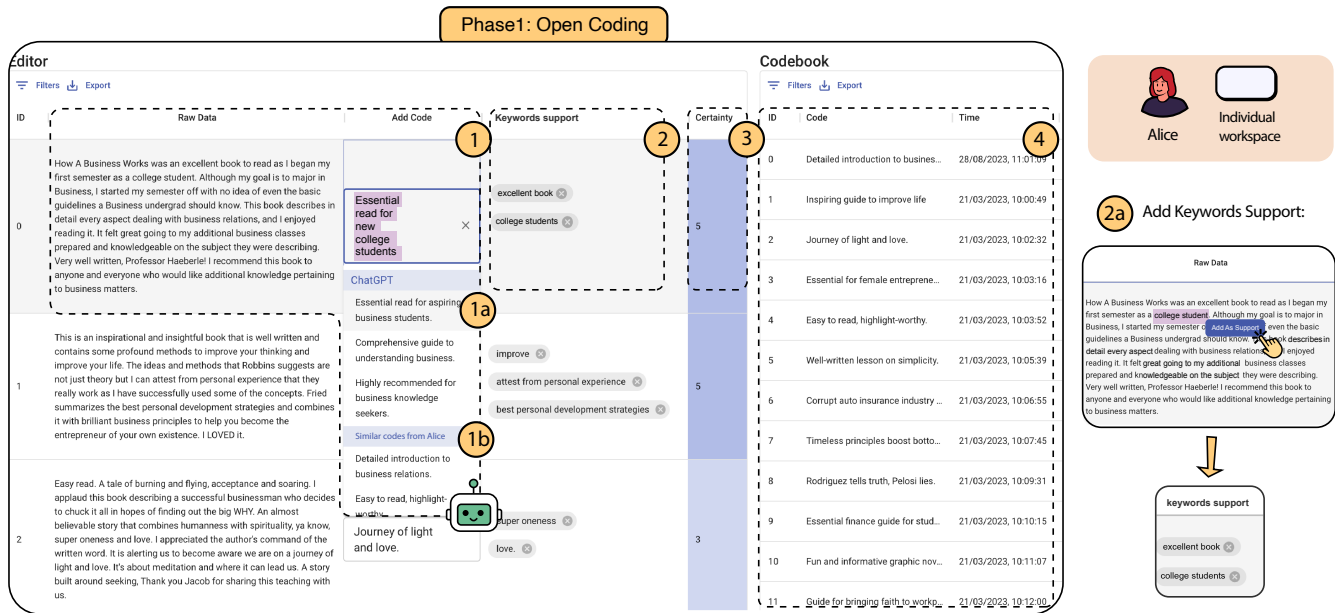


Figure 5: Editing Interface for Phase 1: 1) inputting customized code for the text in "Raw Data", either 1a) choosing from the GPT's recommendations, 1b) choosing from the top three relevant codes; 2) adding keywords support by 2a) selecting from raw data and "Add As Support"; 3) assigning a certainty level ranging from 1 to 5, where 1="very uncertain" and 5="very certain"; and 4) reviewing and modifying the individual codebook.

"college student") from the RAW DATA cell that supports her proposed code, which will be added to the KEYWORDS SUPPORT beside her proposed code. She can also assign a CERTAINTY, ranging from 1 to 5, to the code. This newly generated code will be included in Alice's personal CODEBOOK and can be viewed by her at any time. They can check the progress of each other in the PROGRESS at any time (see Figure 6 1a).

5.1.2 Phase 2: Code Merging and Discussion. Figure 6 depicts the shared workspace where coding teams collaborate, discussing their code choices and making final decisions regarding the codes identified in Phase 1. After completing coding, Alice can check the CHECKBOX next to Bob's name once she sees that his progress is at 100%. Subsequently, she can click the CALCULATE button to generate quantitative metrics such as similarity scores and IRR (Cohen's Kappa and Agreement Rate¹⁴) within the team. The rows are then sorted by similarity scores in descending order.

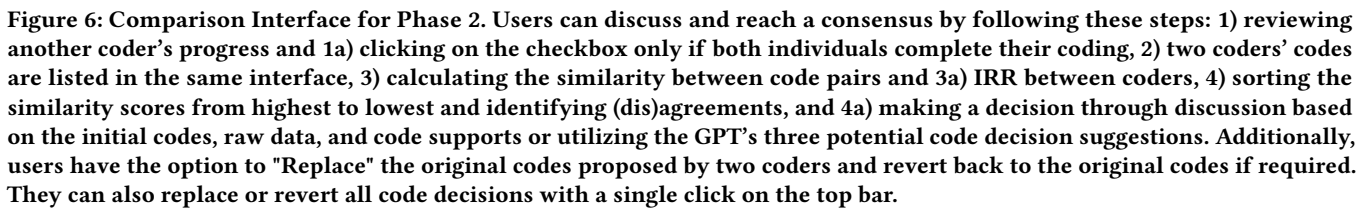
Alice can then share her screen via a Zoom meeting with Bob to COMPARE AND DISCUSS their codes, starting from code pairs with high similarity scores. For instance, Alice's code "Excellent guide for new college students" with a certainty of 5 includes "excellent book" and "college student" supports, while Bob's code "Excellent read for aspiring business students" with a certainty of 4 includes "How A business works" and "as a college student" as KEYWORDS SUPPORT. The similarity score between their codes could be 0.876 (close to 1), showing a high agreement. During the discussion, they both agreed that the

final code should contain the word "student" due to their similar KEYWORDS SUPPORT, but they cannot reach a consensus about the final expression of the code, they then seek GPT suggestions (e.g., "Essential college guide for business students", "Semester's gem for new college students", "Essential college starter"), and decide the final code decision for this unit is "Essential college guide for business students". However, if the code pair presents a low similarity score, they must allocate additional time to scrutinize the code decision information and identify the keywords that led to different interpretations.

Once all code decisions have been made, Alice can then click on REPLACE to replace the original codes, resulting in an update of Cohen's Kappa and Agreement Rate. This action can be undone by clicking on UNDO.

5.1.3 Phase 3: Code Group Generation. Once Alice and Bob have agreed on the final code decisions for all the units, the code decision list will be displayed on the code grouping interface, as shown in Figure 7. This interface is shared uniformly among the coding team. For further discussion, Alice can continue to share her screen with Bob on Zoom. She can hover over each CODE DECISION to refer to the corresponding raw data or double-click to edit. Alice and Bob can collaborate to propose the final code groups by clicking on ADD NEW GROUP and drag the code decisions into the new code group. For instance, a group "Business knowledge" can include "Simplified business knowledge", "Cautionary book on costly Google campaigns" and others. Alternatively, they can request GPT assistance by clicking on the CREATE CODE GROUPS BY AI button to automatically generate several code groups and

¹⁴The calculation methods differ between these two metrics. Cohen's kappa is a more intricate method for measuring agreement, as detailed in [51]. On the other hand, the Agreement Rate represents the percentage of data on which coders concur.



5.2 Key Features

- (1) Editing Interface for Phase 1: Independent Open Coding (Figure 5).
- (2) Comparison Interface for Phase 2: Merge and Discuss (Figure 6).
- (3) Code Group Interface for Phase 3: Code Groups Generation (Figure 7).

5.2.3 Web-based Platform. In alignment with **DG3**, our goal is to harness the synchronization benefits of Atlas.ti Web while preserving the essential independence required for the CQA process.

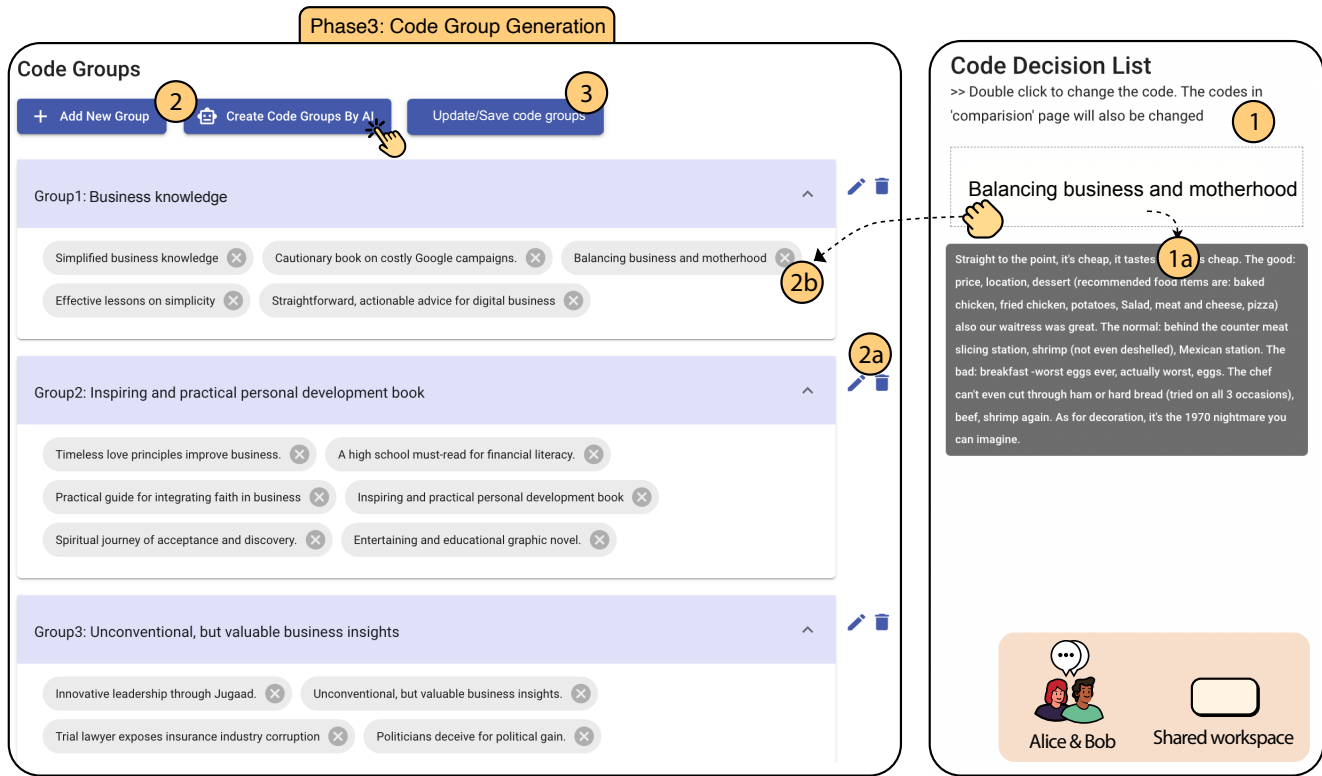


Figure 7: Code Group Interface for Phase 3. It enables users to manage their code decisions in a few steps: 1) the code decisions are automatically compiled into a list of unique codes that users can edit by double-clicking and accessing the original data by hovering over the code. 2) users can group their code decisions by using either "Add New Group" or "Create Code Groups By AI" options. They can then 2a) name or delete a code group or use AI-generated themes, and 2b) drag the code decisions into code groups. 3) Finally, users can save and update the code groups.

CollabCoder addresses this by using a web-based platform. Here, the lead coder creates a project and invites collaborators to engage with the same project. As outlined in section 5.2.2, upon the completion of individual coding, participants can effortlessly view the results of others, eliminating the need for downloading, importing, or further steps.

5.2.4 Consistent Data Units for All Users. Aligned with DG4, our objective is to synchronize coders' interpretation levels to boost discussion efficiency. CollabCoder facilitates this by segmenting data into uniform units (e.g., sentences or paragraphs) that are collaboratively determined by all coders prior to data importation or the onset of coding task.

5.2.5 LLMs-generated Coding Suggestions Once the User Requests. Aligned with DG5, we aim to empower coders to initially develop their own codes and then seek LLMs' assistance when necessary, striking a balance between user autonomy and the advantages of LLMs' support. Apart from proposing their own codes, CollabCoder offers LLMs-generated code suggestions when a user interacts with the input cell. These suggestions appear in a dropdown list for the

chosen data unit after a brief delay (≈ 5 seconds¹⁵), allowing users time to think about their own codes first. At the same time, CollabCoder identifies and provides the three most relevant codes from the current individual codebook for the given text unit, ensuring coding consistency when reusing established codes.

5.2.6 A Shared Workspace for Deeper Discussion. In alignment with DG6, our goal is to establish a shared understanding and foster richer, more substantive discussions. CollabCoder supports this goal through three key features.

- (1) *Documenting Decision-making Rationale.* In Phase 1, CollabCoder allows users to select keywords, phrases, and their coding certainty as supporting evidence. These highlighted elements can represent pivotal factors influencing the user's coding decision. CollabCoder further facilitates users in rating their certainty for each code on a scale from 1 (least certain) to 5 (most certain) to mark the ambiguity.

¹⁵We established a default 2-second delay alongside GPT API's approximate 3-second delay. Investigating the optimal delay is beyond our current research scope, we acknowledge this as a limitation and plan to address it in future research.

- (2) *Side-by-Side Comparison in A Shared Workspace.* Building on DG6's emphasis on establishing common ground, CollabCoder presents all users' coding information for the relevant data units side-by-side. This display includes the original data units, supporting keywords, and indicators of labeled certainty scores. This layout facilitates direct comparison and nuanced discussions.
- (3) *Identifying (Dis)agreements.* CollabCoder simplifies the process of spotting (dis)agreements by calculating the Similarity of the code pair of each unit. This analysis can be executed in 3-10 seconds for all data units. Similarity scores for code pairs range from 0 (low similarity) to 1 (high similarity). For ease of discussion, these scores can be sorted in descending order, with higher scores indicating stronger agreements.

5.2.7 LLMs as a Group Recommender System. In alignment with DG7, our aim is to foster cost-effective and equitable coding outcomes utilizing LLMs. CollabCoder achieves this by serving as an LLM-based group recommender system [39]: when users struggle to finalize a code, CollabCoder proposes three code decision suggestions specific to the code pair, taking into account the raw data, codes from each user, keywords support, and certainty scores. Users can then select and customize these suggestions to reach a conclusive coding decision.

5.2.8 Formation of LLMs-based Code Groups. Consistent with DG8, our objective is to optimize the process of code group creation to enhance efficiency. To this end, CollabCoder introduces the Code Group interface to provide two key functions:

- (1) *Accessing Original Data via the Final Code Decision List.* CollabCoder streamlines final code decisions, presenting them on the right-hand side of the interface. Hovering over a code reveals its originating raw data. Additionally, by double-clicking on an item within the code decision list, users can amend it, and the corresponding codes are updated accordingly.
- (2) *Managing Code Groups.* With CollabCoder, users can effortlessly craft, rename, or delete code groups. They can drag codes from the decision list to a designated code group or remove them. To save users the effort of building groups from scratch, CollabCoder provides an option to enlist GPT's help in organizing code decisions into preliminary groupings. This offers a foundation that users can then adjust, rename, or modify.

5.3 Prompts Design

CollabCoder leverages OpenAI's ChatGPT model (gpt-3.5-turbo)¹⁶ to provide code and code group suggestions. Throughout the three phases, GPT is tasked with the role of "a helpful qualitative analysis assistant", aiding researchers in the development of codes, code decisions, and primary code groups that are crucial for subsequent stages. Additionally, we have tailored different prompts for distinct types of codes. For instance, we use "descriptive codes for raw data" and "relevant codes derived from coding history" (in Phase 1), ensuring a tailored approach for each coding requirement. The prompts, along with the text data, are simultaneously sent to GPT

for processing. All prompts used are listed in Appendix Table 5, 6 and 7. To ensure code suggestions have diversity without being overly random, the temperature parameter is set at 0.7.

5.4 System Implementation

5.4.1 Web Application. The front-end implementation makes use of the react-mui library¹⁷. Specifically, we employed the DataGrid component¹⁸ to construct tables in both the "Edit" and "Compare" interfaces, allowing users to input and compare codes. These tables auto-save user changes through HTTP requests to the backend, storing data in the database to synchronize progress among collaborators. For each data unit, users have their own code, keyword supports, certainty levels, and codebook in the Edit interface, while sharing decisions in the "Compare" interface and code groups in the "Codebook" interface. To prevent users from viewing collaborators' codes before editing is complete, we restrict access to other coders' codes and only show everyone's own progress in the "Compare" interface. We also utilized the foldable Accordion component¹⁹ to efficiently display code group lists in the "Codebook" interface, where users can edit, drag and drop decision objects to modify their code groups. The backend leverages the Express framework, facilitating communication between the frontend and MongoDB. It also manages API calls to the GPT-3.5 model and uses Python to calculate statistics such as similarities.

5.4.2 Data Pre-processing. We partitioned raw data from CSV and txt files into data units during the pre-processing phase. At the sentence level, we segmented the text using common sentence delimiters such as ".", "...", "!", and "?". At the paragraph level, we split the text using `\n\n`.

5.4.3 Semantic Similarity and IRR. In CollabCoder, the IRR is measured using Cohen's Kappa²⁰ and Agreement Rate. To calculate Cohen's Kappa, we used the "cohen_kappa_score" method from scikit-learn package backend²¹. Cohen's Kappa is a score between -1 (total disagreement) and +1 (total agreement). Subsequently, we calculate the Agreement Rate as a score between 0 and 1, by determining the percentage of code pairs whose similarity score exceeds 0.8, indicating that the two coders agree on the code segment. We utilize the *semantic textual similarity* function²² in the *sentence-transformers* package²³ to assess agreements and disagreements in coding. This function calculates the semantic similarity between each code pair from two coders (e.g., Alice: Excellent guide for new college students vs. Bob: Excellent read for aspiring business students) for each data unit. A high similarity score (close to 1) indicates agreement between coders, while a low score (close to 0) suggests disagreement.

¹⁷<https://mui.com/>

¹⁸<https://mui.com/x/react-data-grid/>

¹⁹<https://mui.com/material-ui/react-accordion/>

²⁰Cohen's Kappa is a statistical measure used to evaluate the IRR between two raters, which takes into account the possibility of agreement occurring by chance, thus providing a more accurate representation of agreement than simply calculating the percentage of agreement between the raters.

²¹https://scikit-learn.org/stable/modules/generated/sklearn.metrics.cohen_kappa_score.html

²²https://www.sbert.net/docs/usage/semantic_textual_similarity.html

²³<https://www.sbert.net/>

¹⁶<https://platform.openai.com/docs/models/gpt-3-5>

6 USER EVALUATION

To evaluate CollabCoder and answer our research questions, we conducted a within-subject user study involving 16 (8 pairs) participants who used two platforms: CollabCoder and Atlas.ti Web, for qualitative coding on two sets of qualitative data.

6.1 Participants and Ethics

We invited 16 participants with varying qualitative analysis experiences via public channels and university email lists. We involve both experts and non-experts as lowering the bar is particularly important for newcomers or early researchers who might face significant challenges in adhering to such a rigorous workflow [17, 60]. Among them, 2/16 participants identified as experts, 3/16 considered themselves intermediate, 4/16 as beginners, and 7/16 had no qualitative analysis experience (see details in Appendix Table 8). Participants were randomly matched, leading to the formation of 8 pairs (see Table 3). Each participant received 30 SGD for their participation. The study protocol and the financial compensation at the hourly rate were approved by our university's IRB.

6.2 Datasets

We established two criteria to select the datasets used for the coding task: 1) the datasets should not require domain-specific knowledge for coding, and 2) coders should be able to derive a theme tree and provide insights iteratively. Accordingly, two datasets containing book reviews on "Business" and "History" topics from the Books_v1_00 category of amazon_us_reviews dataset²⁴ were selected. For each of them, we filtered 15 reviews to include only those with a character count between 400 and 700 and removed odd symbols such as \ and
. The workload was determined through pilot tests with several participants.

6.3 Conditions

- **Atlas.ti Web:** a powerful platform for qualitative analysis that enables users to invite other coders to collaborate by adding, editing, and deleting codes. It also allows for merging codes and generating code groups manually.
- **CollabCoder:** the formal version of our full-featured platform.

The presentation order of both platforms and materials was counter-balanced across participants using a Latin-square design [43].

6.4 Procedure

Each study was conducted virtually via Zoom and lasted around 2 to 3 hours. It consisted of a pre-study questionnaire, training for novice participants, two qualitative coding sessions with different conditional systems, a post-study questionnaire, and a semi-structured interview.

6.4.1 Introduction to the Task. After obtaining consent, we introduced the task to the pairs of participants, which involved analyzing reviews and coding them to obtain meaningful insights. We introduced research questions they should take into account when coding, such as recurring themes or topics, common positive and

negative comments or opinions. We provided guidelines to ensure that the coding was consistent across all participants. Participants could use codes up to 10 words long, add similar codes in one cell per data unit, and include both descriptive and in-vivo codes.

6.4.2 Specific Process. Following the introduction, we provided a video tutorial on how to use the platform for qualitative coding. Participants first did independent coding, and then discussed the codes they had found and made final decisions for each unit, ultimately forming thematic groups. We advise coders to first gain a thorough understanding of the text, then seek suggestions from GPT, engage in comprehensive discussions, and finally present code groups that effectively capture the valuable insights they have acquired. To ensure they understood the study purpose better, participants were shown sample code groups as a reference for the type of insights they should aim to obtain from their coding. After completing the coding for all sessions, participants were asked to complete a survey, which included a 5-level Likert Scale to rate the effectiveness of the two platforms, and self-reported feelings towards them.

6.4.3 Data Recording. During the process, we asked participants to share their screens and obtained their consent to record the meeting video for the entire study. Once the coding sessions were completed, participants were invited to participate in a post-study semi-structured interview.

6.4.4 Data analysis. We analyzed interview transcripts and observation notes (see Table 9 and 10) using thematic analysis as described in Braun and Clarke's methodology [6]. After familiarizing ourselves with data and generating initial codes, we grouped the transcripts into common themes derived from the content. Next, we discussed, interpreted, and resolved discrepancies or conflicts during the grouping process. Finally, we reviewed the transcripts to extract specific quotes relevant to each theme. We summarized the following key findings.

7 RESULTS

7.1 RQ1: Can CollabCoder support qualitative coders to conduct CQA effectively?

7.1.1 Key Findings (KF) on features that support CQA.

KF1: CollabCoder workflow simplifies the learning curve for CQA and ensures coding independence in the initial stages. Overall, users found CollabCoder to be better as it supports side-by-side comparison of data, which makes the coding and discussion process **easier to understand (P2)**, **more straightforward (P7)**, and **beginner-friendly (P4)** than Atlas.ti Web, and P4 noted that CollabCoder had a **lower learning curve**.

Moreover, CollabCoder workflow preserves coding independence. Experienced users (P11 and P14), familiar with qualitative analysis, find CollabCoder's independent coding feature to be particularly beneficial: "So you don't see what the other person is coding until like both of you are done. So it doesn't like to affect your own individual coding...[For Atlas.ti Web] the fact like you can see both persons' codes and I think I'm able to edit the other person's codes as well, which I think might not be very a good practice." Similarly, P14 indicated: "I think CollabCoder is better if you aim for independent coding."

²⁴https://huggingface.co/datasets/amazon_us_reviews/viewer/Books_v1_00/train

KF2: Individual workspace with GPT assistance is valued for reducing cognitive burden in Phase 1. CollabCoder makes it easier for beginner users to propose and edit codes compared to Atlas.ti Web. 7/16 participants appreciated that GPT's additional assistance (P7, P15), which gave them reference (P1) and decreased thinking (P9). Such feelings are predominantly reported by individuals who are either beginners or lack prior experience in qualitative analysis. As P13 said, *"I think the CollabCoder one is definitely more intuitive in a sense, because it provides some suggestion, you might not use it, but at least some basic suggestions, whereas the Atlas.ti one, you have to take from scratch and it takes more mental load."*

Some of these beginners also showed displeasure towards GPT, largely stemming from its content summarization level, which users cannot regulate. P1 (beginner) found that **in certain instances, CollabCoder generated highly detailed summaries** which might not be well-suited to their requirements, leading them to prefer crafting their own summaries: *"One is that its summary will be very detailed, and in this case, I might not use its result, but I would try to summarize [the summary] myself."* This caused them to question AI's precision and appropriateness for high-level analysis, especially in the context of oral interviews or focus groups.

In addition, when adding codes, our participants indicated that they preferred **reading the raw data first before looking at the suggestions**, as they believed that reading the suggestions first could influence their thinking process (P1, P3, P4, P14) and introduce bias into their coding: *"So I read the text at first. it makes more sense, because like, if you were to solely base your coding on [the AI agent], sometimes its suggestions and my interpretation are different. So it might be a bit off, whereas if you were to read the text, you get the full idea as to what the review is actually talking about. The suggestion functions as a confirmation of my understanding."* (P4)

KF3: Pre-defined data units, documented decision-making mechanisms, and progress bar features collectively enhance mutual understanding in Phase 2. Regarding collaboration, users found that having a pre-defined unit of analysis enabled them to more easily understand the context: *"I am able to see your quotations. Basically what they coded is just the entire unit. But you see if they were to code the reviews based on sentences, I wouldn't actually do the hard work based on which sentence he highlighted. But for CollabCoder, I am able to see at a glance, the exact quotations that they did. So it gives me a better sense of how their codes came about."* (P3) Moreover, users emphasized the importance of not only having the quotation but also keeping its context using pre-defined data units, as they often preferred to refer back to the original text. This is because understanding the context is crucial for accurate data interpretation and discussion: *"I guess, it is because like we're used to reading a full text and we know like the context rather than if we were to read like short extracts from the text. the context is not fully there from just one or two line [quotations]."* (P9)

Users also appreciated CollabCoder's keywords-support function, as it **aided them in capturing finer details (P9)** and facilitated a deeper understanding of the codes added: *"It presents a clearer view about that paragraph. And then it helps us to get a better idea of what the actual correct code should be. But since the other*

one [Atlas.ti Web] is [...] a little bit more like superficial, because it's based solely on two descriptive words." (P14)

The progress bar feature in CollabCoder was seen as helpful when collaborating with others. It allowed them to **manage their time better and track the progress of each coder**. *"I actually like the progress bar because like that I know where my collaborators are."* (P8) Additionally, it acted as a **tracker to notify the user if they missed out on a part**, which can help to avoid errors and improve the quality of coding. *"So if say, for example, I missed out one of the codes then or say his percentage is at 95% or something like that, then we will know that we missed out some parts"* (P3)

All the above features collectively improve the mutual understanding between coders, which can decrease the effort devoted to revisiting the original data and recalling their decision-making processes, and deepen discussions in a limited time.

KF4: The shared workspace with metrics allows coders to understand disagreements and initiate discussions better in Phase 2. In terms of statistics during the collaboration, the similarity calculation and ranking features enable users to **quickly identify (dis)agreements (P2, P3, P7, P10, P14) to ensure they focus more (P4)**. As P14 said, *"I think it's definitely a good thing [to calculate similarity]. From there, I think we can decide whether it's really a disagreement on whether it's actually two different information."* Moreover, the identification of disagreements is reported to **pave the way for discussion (P1, P8)**: *"So I think in that sense, it just opens up the door for the discussion compared to Atlas.ti...[and] better in idea generation stands and opening up the door for discussion."* (P8) In contrast, Atlas.ti necessitated more discussion initiation on the part of users.

Nevertheless, ranking similarity using CollabCoder might have a negative effect, as it may make coders focus more on improving their agreements instead of providing a more comprehensive data interpretation: *"I think pros and cons. because you will feel like there's a need to get high similarity on every code, but it might just be different codes. So there might be a misinterpretation."* (P7)

The participants had mixed opinions regarding the usefulness of IRR in the coding process. P9 found Cohen's kappa useful for their report as they do not need to calculate manually: *"I think it's good to have Cohen's Kappa, because we don't have to manually calculate it, and it is very important for our report."* However, P6 did not consider the statistics to be crucial in their personal research as they usually do coding for interview transcripts. *"Honestly, it doesn't really matter to me because in my own personal research, we don't really calculate. Even if we have disagreements, we just solve it out. So I can't comment on whether the statistics are relevant, right from my own personal experience."* (P6)

KF5: The GPT-generated primary code groups in Phase 3 enable coders to have a reference instead of starting from scratch, thereby reducing cognitive burden. Participants expressed a preference for the automatic grouping function of CollabCoder, as it was **more efficient (P1, P2, P8, P14) and less labor-intensive (P3)**, compared to the more manual approach in Atlas.ti Web. In particular, P14 characterized the primary distinction between the two platforms as Atlas.ti Web adopts a "bottom-up approach" while CollabCoder employs a "top-down approach". In this context,

the "top-down approach" refers to the development of "overall categories/code groups" derived from the coding decisions made in Phase 2, facilitated by GPT. This approach allows users to modify and refine elements within an established primary structure or framework, thereby eliminating the need to start from scratch. Conversely, the "bottom-up approach" means generating code groups from an existing list, through a process of reviewing, merging, and grouping codes with similar meanings. This difference impacts the mental effort required to create categories and organize codes. *"I think it's different also because Atlas.ti is more like a bottom-top approach. So we need to see through the primary codes to create the larger categories which might be a bit more tedious, because usually, they are the primary codes. So it's very hard to see an overview of everything at once. So it takes a lot of mental effort, but for CollabCoder, it is like a top-down approach. So they [AI] create the overall categories. And then from there, you can edit and then like shift things around which helps a lot. So I also prefer CollabCoder."* (P14) P1 also highlighted that this is particularly helpful when dealing with large amounts of codes, as manually grouping them one-by-one becomes nearly unfeasible.

7.1.2 Key Findings (KF) on collaboration behaviors with CollabCoder supports.

KF6: An analysis of three intriguing group dynamics manifested in two conditions. In addition to the key findings on feature utilization, we observed three intriguing collaboration group dynamics, including "follower-leader" (P1×P2, P5×P6), "amicable cooperation" (P3×P4, P7×P8, P9×P10, P13×P14, P15×P16) and "swift but less cautious" (P11×P12). The original observation notes are listed in Appendix Table 9 and 10.

The "follower-leader" pattern typically occurred when one coder was a novice, while the other had more expertise. Often, the inexperienced coder contributed fewer ideas or only offered support during the coding process: when using Atlas.ti Web, those "lead" coders tended to take on more coding tasks than the others since their coding tasks could not be precisely quantified. Even though both of them were told to code all the data, it would end up in a situation where one coder primarily handled the work while the other merely followed with minimal input. This pattern could also appear if the coders worked at different paces (P1×P2). As a result, the more efficient coders expressed more ideas. In contrast, CollabCoder ensures equitable participation by assigning the same coding workload to each participant and offering detailed documentation of the decision-making process via its independent coding interface. This approach guarantees that coders, even if they seldom voice their opinions directly, can still use the explicit documented information to communicate their ideas indirectly and be assessed in tandem with their collaborators. Furthermore, the suggestions generated by GPT are derived from both codes and raw data, producing a similar effect.

For "amicable cooperation", the coders respected each other's opinions while employing CollabCoder as a collaborative tool to finalize their coding decisions. When they make a decision, they firstly identify the common keywords between their codes, and then check the suggestions with similar keywords to decide whether to use suggestions or propose their own final code decision. Often, they took turns to apply the final code. For example, for the first

data unit, one coder might say, *"hey, mine seems better, let's use mine as the final decision,"* and for the second one, the coder might say, *"hey, I like yours, we should choose yours [as the final decision]"* (P3×P4). In some cases, such as P13×P14, both coders generally reach a consensus, displaying no strong dominance and showing respect for each other's opinions, sometimes it is challenging to finalize a terminology for the code decision. Under this kind of condition, the coders used an LLMs agent as a mediator to find a more suitable expression that takes into account both viewpoints. Although most groups maintained similar "amicable cooperation" dynamics in Atlas.ti Web sessions, some found it challenging to adhere to their established patterns. This difficulty is attributed to the fact that such patterns are more resource-intensive. Take the P7×P8 scenario as an example: in this case, the participants encountered time management challenges, as each coding session was initially scheduled to conclude within half an hour. Participants were afforded some flexibility, allowing sessions to extend slightly beyond the initially planned duration to ensure the completion of their tasks. In the CollabCoder condition, they engaged in extensive and respectful discussions, which consequently reduced the time available for the Atlas.ti Web session. Consequently, they had to expedite the process in Atlas.ti Web. This rush resulted in a situation where only one coder assumed the responsibility of merging the codes and rapidly grouping them into thematic clusters. For this coder, to access deeper insights behind these codes, additional operations like asking why another coder has this code, and clicking more to understand which sentence it means were often not feasible within the time constraints. This absence of operations forced coders to merge data relying solely on codes, without the advantage of additional contextual insights. Consequently, this approach often leads to a "follower-leader" or "leader-takes-all" dynamic. While this simplifies the process for participants, it potentially compromises the quality of the discussion. This is also evidenced by our quantitative data in Table 3.

The "swift but less cautious" collaboration was a less desirable pattern we noticed: For P11×P12, during the merging process, they would heavily rely on GPT-generated decisions in order to finish the task quickly. This scenario highlights the concerns regarding excessive reliance on GPT and insufficient deep thinking, which can negatively impact the final quality even when GPT is used as a mediator after the codes have been produced, as defined as our initial objective. Under this pattern, the pair sadly used GPT for "another round of coding" rather than as a neutral third-party decision advice provider. In the case of this particular pair working with Atlas.ti Web, a distinct pattern emerged: P11 exhibited a notably faster pace, while P12 worked more slowly. As a result, the collaboration between the participants evolved into a "follower-leader" dynamic. In this structure, the quicker participant, P11, appeared to steer the overall process, occasionally soliciting inputs from P12.

7.2 RQ2. How does CollabCoder compare to currently available tools like Atlas.ti Web?

7.2.1 Post-study questionnaire. We gathered the subjective preferences from our participants. To do so, we gave them 12 statements like *"I find it effective to..."* and *"I feel confident/prefer..."* pertaining to the effectiveness and self-perception. We then asked them to

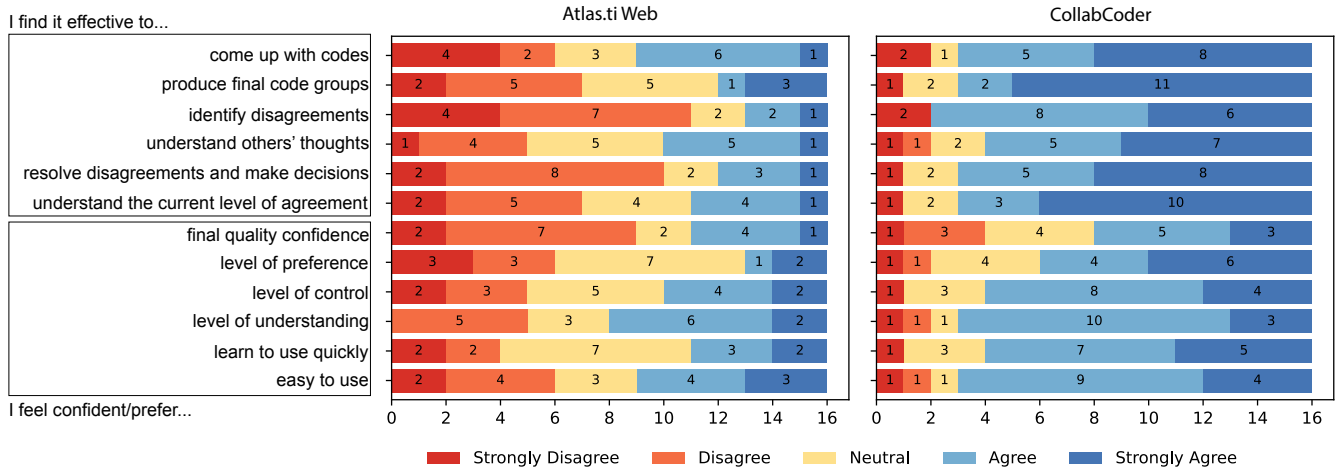


Figure 8: Post-study Questionnaires Responses from Our Participants on Different Dimensions on A 5-point Likert Scale, where 1 denotes "Strongly Disagree", 5 denotes "Strongly Agree". The numerical values displayed on the stacked bar chart represent the count of participants who assigned each respective score.

rate their agreement with each sentence on a 5-point Likert scale for each platform. The details of the 12 statements are shown in Figure 8.

Overall, pairwise t-tests showed that participants rated CollabCoder significantly (all $p < .05$) better than Atlas.ti Web for effectiveness in 1) *coming up with codes*, 2) *producing final code groups*, 3) *identifying disagreements*, 4) *resolving disagreements and making decisions*, 5) *understanding the current level of agreement*, and 6) *understanding others' thoughts*. The results also indicated that participants believed CollabCoder ($M = 4$) could be learned for use quickly compared to Atlas.ti Web ($M = 3.1$, $t(15) = -3.05$, $p < .01$). For other dimensions, the *confidence in the final quality*, *perceived level of preference*, *level of control*, *level of understanding*, and *ease of use*, while our results show a general trend where CollabCoder achieves higher scores, we found no significant differences. Additionally, we observed that one expert user (P6) exhibited a highly negative attitude towards implementing AI in qualitative coding, as he selected "strongly disagree" for nearly all the assessment criteria. We will discuss his qualitative feedback in Section 8.2.3.

7.2.2 Log data analysis. A two-tailed pairwise t-test on *Discussion Time* revealed a significant difference ($t(15) = -3.22$, $p = .017$) between CollabCoder ($M \approx 24mins$, $SD \approx 7mins$) and Atlas.ti Web ($M \approx 11mins$, $SD \approx 5.5mins$). **Discussions under the CollabCoder condition were significantly longer than those in the Atlas.ti Web condition.** When examining the IRR, it was found that the IRRs in the Atlas.ti Web condition were overall significantly ($t(7) = -6.69$, $p < .001$) lower ($M = 0.06$, $SD = 0.40$), compared to the CollabCoder condition ($M \approx 1$). In the latter, participants thoroughly examined all codes, resolved conflicts, merged similar codes, and reached a final decision for each data unit. Conversely, Atlas.ti Web posed challenges in comparing individual data units side-by-side, leading to minimal code discussions overall (averaging 4.5 codes discussed) compared to the CollabCoder option (averaging 15 codes discussed). Consequently, we surmise that concealed

disagreements within Atlas.ti Web might require additional discussion rounds to attain a higher agreement level. Further evidence is needed to validate this assumption.

7.3 RQ3. How can the design of CollabCoder be improved?

While CollabCoder effectively facilitates collaboration in various aspects, as discussed in Section 7.1, we observed divergent attitudes toward certain functions, such as labeling certainty, relevant code suggestions, and the use of individual codebooks.

Most participants expressed concerns about the clarity, usefulness, and importance of the certainty function in CollabCoder. The self-reported nature, the potential of inconsistencies in reporting, and minimal usage among users suggest that the **certainty function may not be as helpful as intended**. For example, P12 found the certainty function "not really helpful", and P13 admitted forgetting about it due to the numerous other subtasks in the coding process. P3 also reported limited usage of the function, mainly assigning low certainty scores when not understanding the raw data. However, P14 recognized that the certainty function could be helpful in larger teams, as it might help flag quotes that require more discussion.

The perceived usefulness of the relevant code function in CollabCoder depends on the dataset and users' preferences. Some participants found it **less relevant than the AI agent's summary function**, which they considered more accurate and relevant. "Maybe not that useful, but I think it depends on your dataset. Say whether they are many similar data points or whether they are different data points. So I think in terms of these cases they are all very different, have a lot of different contents. So it's not very relevant, but definitely, I think, in datasets which might be more relevant, could be useful." (P2)

As for the individual codebook function, although users acknowledged its potential usefulness in tracking progress and handling

Table 3: Overview of the final coding results. "Collab." denotes CollabCoder, "Atlas." denotes Atlas.ti Web, "Total Codes" denotes the total number of codes generated while "Discussed Codes" denotes the total number of codes that were discussed by the coders during the discussion phase. "Bus." denotes the "Business" dataset while "His." denotes the "History" dataset. "Suggestions Acceptance" column denotes the proportion of usage of GPT-generated codes (GPT), the selection from the relevant codes in code history suggested by GPT (Rele.), and users' self-proposed codes (Self.) to the total number of open codes in Phase 1. "GPT-based Code Decisions" column reflects the proportion of code decisions in Phase 2 that originated from suggestions made by the GPT mediator.

Pairs	Self-reported QA expertise	Conditions	Collaboration Observation	Total Codes		Discussed Codes (No.)		IRR (-1 to 1)		Code Groups (No.)		Discussion Time (mins:secs)		Suggestions Acceptance in Phase 1 (%)			GPT-Based Code Decisions (%)
				Collab.	Atlas.	Collab.	Atlas.	Collab. ^b	Atlas.	Collab.	Atlas.	Collab.	Atlas.	GPT	Rele.	Self.	
P1	Beginner	Atlas. (Bus.), Collab. (His.)	Followers-Leader	15	24	15	6	NA	-0.07	6	3	19:41	07:39	100	0	0	5
P2	No Experience													70	5	25	
P3	Expert	Collab. (Bus.), Atlas. (His.)	Amicable Cooperation	15	10	15	10	NA	1	5	4	35:24	21:32	90	0	10	40
P4	No Experience													90	0	10	
P5	No Experience	Atlas. (His.), Collab. (Bus.)	Followers-Leader	15	11	15	2	NA	-0.02	5	2	17:55	06:16	73	7	20	100
P6	Expert													100	0	0	
P7	No Experience	Collab. (His.)	Amicable Cooperation	15	22	15	2	NA	-0.33	7	6	29:08	No discussion ^a	7	0	93	80
P8	No Experience	Atlas. (Bus.)												13	7	80	
P9	Intermediate	Atlas. (Bus.), Collab. (His.)	Amicable Cooperation	15	17	15	5	NA	0.04	5	2	15:11	14:38	73	13	13	80
P10	No Experience													53	40	7	
P11	Intermediate	Collab. (Bus.), Atlas. (His.)	Quick and not careful	15	61	15	2	NA	-0.07	3	3	19:23	14:15	100	0	0	100
P12	No experience													100	0	0	
P13	Beginner	Atlas. (His.), Collab. (Bus.)	Amicable Cooperation	15	30	15	5	NA	-0.08	8	2	29:19	08:43	87	7	7	100
P14	Intermediate													93	0	7	
P15	Beginner	Collab. (His.)	Amicable Cooperation	15	8	15	4	NA	0.04	4	2	29:09	08:52	100	0	0	43
P16	No experience	Atlas. (Bus.)												73	20	7	
Mean				15	22.88	15	4.5	NA	0.06	5.38	3	24:00	10:48	76.46	6.15	17.4	68.5
SD				0	17.19	0	2.73	NA	0.40	1.60	1.41	07:12	05:24	29.43	10.74	28.11	35.3

^a P7 and P8 gave up discussion for the Atlas.ti session due to spending too much time in the CollabCoder session.

^b Following the discussion session in CollabCoder, the original codes have been restructured and finalized as a single code decision, resulting in an IRR≈1. Consequently, IRR calculations are not applicable (NA) for the CollabCoder conditions.

large datasets, most users *"did not pay much attention to it during this coding process"* (P2, P3, P4). P3 found it helpful for tracking progress but did not pay attention to it during this particular process. P4 acknowledged that the function could be useful in the long run, particularly when dealing with a large amount of data.

While these features may not be as useful as initially anticipated, evidenced by low usage frequency or varying effectiveness across different datasets, further investigation is necessary to ascertain if the needs and challenges associated with these features truly exist or are merely perceived by us. This could significantly enhance user experiences with CollabCoder and inform the future design of AI-assisted CQA tools.

8 DISCUSSION AND DESIGN IMPLICATIONS

8.1 Facilitating Rigorous, Lower-barrier CQA Process through Workflow Design Aligned with Theories

Practically, CollabCoder contributes by providing a one-stop, end-to-end workflow that ensures seamless data transitions between stages with minimal effort. This design is grounded in qualitative analysis theories such as Grounded Theory [24] and Thematic Analysis [45], as outlined in Section 2, facilitating a rigorous yet accessible approach to CQA practice. While spreadsheets are also capable of similar processes, they typically demand considerable effort and struggle to uphold a stringent process due to the intricacy and nuances involved. CollabCoder, in contrast, streamlines these tasks, rendering the team coordination process [22, 47] more practical and manageable. Our evaluation demonstrates the effectiveness of CollabCoder, empowering both experienced practitioners and

novices to perform rigorous and comprehensive qualitative analysis.

Apart from practical benefits, our CollabCoder design [9] can also enrich theoretical understanding in the CQA domain [41], which aids practitioners in grasping foundational theories, thereby bolstering the credibility of qualitative research [14, 41]. Over the years, CQA practices have remained inconsistent and vague, particularly regarding when and how multiple coders may be involved, the computation of IRR, the use of individual coding phases, and adherence to existing processes [5, 54]. A common question could arise: *"If deviating from strict processes does not significantly impact results, or the influence is hard to perceive (at least from others' perspective), why should substantial time be invested in maintaining them, especially under time constraints?"* Current software like Atlas.ti, MaxQDA often neglects this critical aspect in their system design, focusing instead on basic functionalities like data maintenance and code addition, which, are not the most challenging parts of the process for practitioners. Ultimately, CollabCoder enables practitioners to conduct a CQA process that is both transparent and standardized within the community [52, 54]. Looking forward, we foresee a future where coders, in documenting their methodologies, will readily reference their use of such specifically designed workflows or systems for CQA analysis.

With this in mind, our objective is not to position any single method as the definitive standard in this field. Although CollabCoder is specifically designed for one type of coding — consensus coding within inductive coding — we do not exclusively advocate for either consensus or split coding. Instead, we emphasize that coders should choose a method that aligns best with their data and requirements [14, 32, 41, 64]. Therefore, the design of such tools

should aim to accommodate various types of qualitative analysis methods. For instance, split coding might necessitate distributing data among team members in a manner that differs from the uniform distribution required by consensus coding.

8.2 LLMs as “Suggestion Provider” in Open Coding: Helper, not Replacement.

8.2.1 Utilizing LLMs to Reduce Cognitive Burden. Independent open coding is a highly cognitively demanding task, as it requires understanding the text, identifying the main idea, creating a summary based on research questions, and formulating a suitable phrase to convey the summary [15, 43]. Additionally, there is the need to refer to and reuse previously created codes. In this context, GPT’s text comprehension and generation capabilities can assist in this mentally challenging process by serving as a suggestion provider.

8.2.2 Improving LLMs’ Suggestions Quality. However, a key consideration according to KF2 is how GPT can provide better quality suggestions that align with the needs of users. For CollabCoder, we only provided essential prompts such as “summary” and “relevant codes”. However, a crucial aspect of qualitative coding is that coders should always consider their research questions while coding and work towards a specific direction. For instance, are they analyzing the main sentiment of the raw data or the primary opinion regarding something? This factor can significantly impact the coding approach (e.g., descriptive or in-vivo coding [62]) and what should be coded (e.g., sentiment or opinions). Therefore, the system should support mechanisms for users to inform GPT of the user’s intent or direction. One possible solution is to include the research question or intended direction in the prompt sent to GPT alongside the data to be coded. Alternatively, users could configure a customized prompt for guidance, directing GPT’s behavior through the interface [37]. This adaptability accommodates individual preferences and improves the overall user experience.

Looking ahead, as the underlying LLM evolves, we envision that an approach for future LLM assistance in CollabCoder involves: 1) creating a comprehensive library of both pre-set and real-time updated prompts, designed to assist in suggesting codes across diverse fields like psychology and HCI; 2) implementing a feature that allows coders to input custom prompts when the default prompts are not suitable.

8.2.3 LLMs should Remain a Helper. Another key consideration is how GPT can stay a reliable suggestion provider without taking over from the coder [40, 49]. Our study demonstrated that both novices and experts valued GPT’s assistance, as participants used GPT’s suggestions either as code or as a basis to create codes 76.67% of the time on average.

However, one expert user (P6) held a negative attitude towards employing LLMs in open coding, assigning the lowest score to nearly all measures (see Figure 8). This user expressed concerns about the role of AI in this context, suggesting that qualitative researchers might feel forced to use AI-generated codes, which could introduce potential biases. Picking up the nuances from the text is considered “fun” for qualitative researchers (P6), and suggestions should not give the impression that “the code is done for them and

they just have to apply it” (P6) or lead them to “doubt their own ideas” (P5).

On the other side, it is important not to overlook the risk of over-reliance on GPT. While we want GPT to provide assistance, we do not intend for it to fully replace humans in the process, as noted in DG5. Our observations revealed that although participants claimed they would read the raw data first and then check GPT’s suggestions, some beginners tended to rely on GPT for forming their suggestions, and experts would unconsciously accept GPT’s suggestions if unsure about the meaning of the raw data, in order to save time. Therefore, preserving the enjoyment of qualitative research and designing for appropriate reliance [44] to avoid misuse [19] or over-trust can be a complex challenge [67]. To this end, mixed-initiative systems [1, 35] like CollabCoder can be designed to allow for different levels of automation. For example, GPT-generated suggestions could be provided only for especially difficult cases upon request, rather than being easily accessible for every unit, even when including a pre-defined time delay.

8.3 LLMs as “Mediator” and “Facilitator” in Coding Discussion

Among the three critical CQA phases we pinpointed, aside from the open coding phase, the subsequent two stages — Phase 2 (merge and discussion) and Phase 3 (development of a codebook) — require a shared workspace for coders to converse. We took note of the role LLMs undertook during these discussions.

8.3.1 LLMs as a “Mediator” in Group Decision-Making. The challenge of dynamically reaching consensus — a decision that encapsulates the perspectives of all group members — has garnered attention in the HCI field [21, 40, 59]. Jiang et al. [40] extensively explore collaborative dynamics in their research for qualitative analysis. They highlighted decision-making modes in consensus-building may vary under different power dynamics [36] in CQA context. In some cases, the primary author or a senior member of a project may assume the decision-making role. According to our KF6, we also found interesting group dynamics, identifying patterns like “amicable cooperation”, “follower-leader”, and “swift but less cautious” modes. Our design positions GPT as a mediator or a group recommendation system [21], particularly useful when consensus is hard to reach. In this role, GPT acts as an impartial facilitator, aiding in harmonizing labor distribution and opinion expression. It guides groups towards decisions that are not only cost-effective but also equitable, justified, and sound [11]. This is a functionality that can hardly be achieved by using tools like Atlas.ti Web. Moreover, these group dynamics can be explored through various lenses, such as the Thomas-Kilmann conflict modes [65], which emphasize the importance of balancing assertiveness and cooperativeness in a team. Delving into these theories can significantly aid in the design of more effective team collaboration tools.

Nonetheless, CollabCoder’s present design in Phase 2, which employs LLMs as a recommendation system for coding decisions, represents merely an initial step. While the CollabCoder cannot fundamentally alter collaborative power dynamics, it ensures that coding is a collaborative effort, emphasizing substantive discussions between two coders to avoid superficial collaboration. Looking ahead, there are numerous paths we could and should pursue.

For example, as humans should be the ultimate decision-makers, with GPT serving merely as a fair mediator between coders, group decision recommendations ought to be made available only upon explicit request. Alternatively, once a coder puts forth a final decision, GPT could then refine the wording or formulate some conclusive description to facilitate future reflection on the code decisions [3].

8.3.2 LLMs as “Facilitator” in Streamlining Primary Code Grouping. As per KF5, our participants offered insightful feedback about using GPT to generate primary code groups. They found the top-down approach, where GPT first generates primary groups and users subsequently refine and revise them, more efficient and less cognitively demanding compared to the traditional bottom-up method. In the traditional method, users must begin by examining all primary codes, merging them, and then manually grouping them into categories, which can be mentally taxing. Differently, CollabCoder is designed to initially formulate primary or coarse ideas about how to group codes. Similar to many types of recommendation systems, the suggestions provided by CollabCoder are intended to complement the coders’ initial thoughts on code grouping. When coders review these GPT-suggested code groups, it enables them to reflect upon and compare their own ideas with the given suggestions. This process enriches the final code groups by efficiently incorporating a wider range of perspectives, extending beyond the insights of just the two coders. This ensures a more comprehensive and multifaceted categorization. Moreover, researchers can more effectively and easily manage large volumes of data and potentially enhance the quality of their analysis.

However, it is crucial to exercise caution when applying this method. We observed that when time constraints exist, coders may skip discussions, with only one of two coders combining and categorizing the codes into code groups (P7×P8). Additionally, P14 mentioned that GPT appears to dominate the code grouping process, resulting in a single approach to grouping. For instance, while the participants might create code groups based on sentiment analysis during their own coding process, they could be tempted to focus on content analysis under GPT’s guidance.

Similarly, to overcome these challenges of CollabCoder, we envision a system where coders would create their own groupings first and only request LLMs’ suggestions afterward. Alternatively, LLMs’ assistance could be limited to situations where the data volume is substantial. Another approach could be prompting LLMs to generate code groups based on the research questions rather than solely on the (superficial) codes. This would ensure a more contextually relevant and research-driven code grouping process.

9 LIMITATIONS AND FUTURE WORK

This work has limitations. **Firstly, it’s important to note that the current version of CollabCoder operates under certain assumptions, deeming coding tasks as “ideal” — comprising semantically independent units, a two-person coding team, and data units with singular semantics.** However, our expert interviews revealed a more complex reality. One primary source of disagreement arises when different users assign multiple codes to the same data unit, often sparking discussions during collaborative coding. Future research should aim to address this point.

Secondly, we only used pre-defined unit data and did not consider splitting complex data into units (e.g., interview data). Future work could explore utilizing GPT to support the segmentation of interview data into semantic units and automating the import process.

Lastly, we did not investigate the specific process by which users select and edit a GPT suggestion. Future research could delve deeper into how users incorporate these suggestions to generate a final idea. The optimal time for balancing user autonomy and appropriate reliance should also be explored. Moreover, for a tool that could be used by the same coder on multiple large datasets, it would also be beneficial to have GPT generate suggestions based on users’ coding patterns rather than directly providing suggestions.

10 CONCLUSION

This paper introduces CollabCoder, a system that integrates the key stages of the CQA process into a one-stop workflow, aiming to lower the bar for adhering to a strict CQA procedure. Our evaluation with 16 participants indicated a preference for CollabCoder over existing platforms like Atlas.ti Web due to its user-friendly design and GPT assistance tailored for different stages. We also demonstrated the system’s capability to streamline and facilitate discussions, guide consensus-building, and create codebooks. By examining both human-AI and human-human interactions within the context of qualitative analysis, we have uncovered key challenges and insights that can guide future design and research.

ACKNOWLEDGMENTS

We express our gratitude to the anonymous reviewers for their valuable insights, which have significantly improved our paper.

REFERENCES

- [1] J.E. Allen, C.I. Guinn, and E. Horvitz. 1999. Mixed-initiative interaction. *IEEE Intelligent Systems and their Applications* 14, 5 (1999), 14–23. <https://doi.org/10.1109/5254.796083>
- [2] Ross C Anderson, Meg Guerreiro, and Joanna Smith. 2016. Are all biases bad? Collaborative grounded theory in developmental evaluation of education policy. *Journal of Multidisciplinary Evaluation* 12, 27 (2016), 44–57. <https://doi.org/10.56645/jmde.v12i27.449>
- [3] Christine A. Barry, Nicky Britten, Nick Barber, Colin Bradley, and Fiona Stevenson. 1999. Using Reflexivity to Optimize Teamwork in Qualitative Research. *Qualitative Health Research* 9, 1 (1999), 26–44. <https://doi.org/10.1177/1049732999129121677> arXiv:<https://doi.org/10.1177/1049732999129121677> PMID: 10558357.
- [4] Pernille Bjørn, Morten Esbensen, Rasmus Eskild Jensen, and Stina Matthiesen. 2014. Does Distance Still Matter? Revisiting the CSCW Fundamentals on Distributed Collaboration. *ACM Trans. Comput.-Hum. Interact.* 21, 5, Article 27 (nov 2014), 26 pages. <https://doi.org/10.1145/2670534>
- [5] Jana Bradley. 1993. Methodological issues and practices in qualitative research. *The Library Quarterly* 63, 4 (1993), 431–449.
- [6] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101. <https://doi.org/DOI:10.1191/1478088706qp0630a>
- [7] Antony Bryant and Kathy Charmaz. 2007. *The Sage handbook of grounded theory*. Sage.
- [8] Courttni Byun, Piper Vasicek, and Kevin Seppi. 2023. Dispensing with Humans in Human-Computer Interaction Research. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI EA '23). Association for Computing Machinery, New York, NY, USA, Article 413, 26 pages. <https://doi.org/10.1145/3544549.3582749>
- [9] Philip J. Cash. 2018. Developing theory-driven design research. *Design Studies* 56 (2018), 84–119. <https://doi.org/10.1016/j.destud.2018.03.002>
- [10] Kathy Charmaz. 2014. *Constructing grounded theory*. sage.
- [11] Li Chen, Marco de Gemmis, Alexander Felfernig, Pasquale Lops, Francesco Ricci, and Giovanni Semeraro. 2013. Human Decision Making and Recommender

- Systems. *ACM Trans. Interact. Intell. Syst.* 3, 3, Article 17 (oct 2013), 7 pages. <https://doi.org/10.1145/2533670.2533675>
- [12] Nan-Chen Chen, Margaret Drouhard, Rafal Kocielnik, Jina Suh, and Cecilia R. Aragon. 2018. Using Machine Learning to Support Qualitative Coding in Social Science: Shifting the Focus to Ambiguity. *ACM Trans. Interact. Intell. Syst.* 8, 2, Article 9 (jun 2018), 20 pages. <https://doi.org/10.1145/3185515>
 - [13] Herbert H. Clark and Susan E. Brennan. 1991. Grounding in Communication. In *Perspectives on Socially Shared Cognition*, Lauren Resnick, Levine B., M. John, Stephanie Teasley, and D. (Eds.). American Psychological Association, 13–1991. <https://doi.org/10.1037/10096-006>
 - [14] Christopher S. Collins and Carrie M. Stockton. 2018. The Central Role of Theory in Qualitative Research. *International Journal of Qualitative Methods* 17, 1 (2018), 1609406918797475. <https://doi.org/10.1177/1609406918797475> arXiv:<https://doi.org/10.1177/1609406918797475>
 - [15] Juliet Corbin and Anselm Strauss. 2008. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Sage publications Sage.
 - [16] Juliet M Corbin and Anselm Strauss. 1990. Grounded theory research: Procedures, canons, and evaluative criteria. *Qualitative sociology* 13, 1 (1990), 3–21. <https://doi.org/10.1007/BF00988593>
 - [17] Flora Cornish, Alex Gillespie, and Tania Zittoun. 2013. Collaborative analysis of qualitative data. *The SAGE handbook of qualitative data analysis* 79 (2013), 93. <https://doi.org/10.4135/9781446282243>
 - [18] Margaret Drouhard, Nan-Chen Chen, Jina Suh, Rafal Kocielnik, Vanessa Pena-Araya, Keting Cen, Xiangyi Zheng, and Cecilia R Aragon. 2017. Aeonium: Visual analytics to support collaborative qualitative coding. In *2017 IEEE Pacific Visualization Symposium (PacificVis)*. IEEE, 220–229.
 - [19] Mary T Dzindolet, Scott A Peterson, Regina A Pomranky, Linda G Pierce, and Hall P Beck. 2003. The role of trust in automation reliance. *International journal of human-computer studies* 58, 6 (2003), 697–718.
 - [20] Jessica Diaz, Jorge Pérez, Carolina Gallardo, and Ángel González-Prieto. 2023. Applying Inter-Rater Reliability and Agreement in collaborative Grounded Theory studies in software engineering. *Journal of Systems and Software* 195 (2023), 111520. <https://doi.org/10.1016/j.jss.2022.111520>
 - [21] Hanif Emamgholizadeh. 2022. Supporting Group Decision-Making Processes Based on Group Dynamics. In *Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization (Barcelona, Spain) (UMAP '22)*. Association for Computing Machinery, New York, NY, USA, 346–350. <https://doi.org/10.1145/3503252.3534358>
 - [22] Elliot E Entin and Daniel Serfaty. 1999. Adaptive team coordination. *Human factors* 41, 2 (1999), 312–325.
 - [23] Jessica L. Feuston and Jed R. Brubaker. 2021. Putting Tools in Their Place: The Role of Time and Perspective in Human-AI Collaboration for Qualitative Analysis. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW2, Article 469 (oct 2021), 25 pages. <https://doi.org/10.1145/3479856>
 - [24] Uwe Flick. 2013. *The SAGE handbook of qualitative data analysis*. Sage.
 - [25] Abbas Ganji, Mania Orand, and David W. McDonald. 2018. Ease on Down the Code: Complex Collaborative Qualitative Coding Simplified with 'Code Wizard'. *Proc. ACM Hum.-Comput. Interact.* 2, CSCW, Article 132 (nov 2018), 24 pages. <https://doi.org/10.1145/3274401>
 - [26] Jie Gao, Kenny Tsu Wei Choo, Junming Cao, Roy Ka-Wei Lee, and Simon Perrault. 2023. CoAlCoder: Examining the Effectiveness of AI-Assisted Human-to-Human Collaboration in Qualitative Analysis. *ACM Trans. Comput.-Hum. Interact.* (aug 2023). <https://doi.org/10.1145/3617362> Just Accepted.
 - [27] Simret Araya Gebreegziabher, Zheng Zhang, Xiaohang Tang, Yihao Meng, Elena L. Glassman, and Toby Jia-Jun Li. 2023. PaTAT: Human-AI Collaborative Qualitative Coding with Explainable Interactive Rule Synthesis. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (Hamburg, Germany) (CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 362, 19 pages. <https://doi.org/10.1145/3544548.3581352>
 - [28] Linda S. Gilbert, Kristi Jackson, and Silvana di Gregorio. 2014. *Tools for Analyzing Qualitative Data: The History and Relevance of Qualitative Data Analysis Software*. Springer New York, New York, NY, 221–236. https://doi.org/10.1007/978-1-4614-3185-5_18
 - [29] Barney Glaser and Anselm Strauss. 2017. *Discovery of grounded theory: Strategies for qualitative research*. Routledge.
 - [30] Jamie C Gorman. 2014. Team coordination and dynamics: two central issues. *Current Directions in Psychological Science* 23, 5 (2014), 355–360.
 - [31] Jamie C Gorman, Polemnia G Amazeen, and Nancy J Cooke. 2010. Team coordination dynamics. *Nonlinear dynamics, psychology, and life sciences* 14, 3 (2010), 265.
 - [32] Grad Coach. 2023. Qualitative Data Analysis Methods: Top 6 + Examples. <https://gradcoach.com/qualitative-data-analysis-methods/>.
 - [33] Wendy A Hall, Bonita Long, Nicole Bernbach, Sharalyn Jordan, and Kathryn Patterson. 2005. Qualitative teamwork issues and strategies: Coordination through mutual adjustment. *Qualitative Health Research* 15, 3 (2005), 394–410.
 - [34] Matt-Heun Hong, Lauren A. Marsh, Jessica L. Feuston, Janet Ruppert, Jed R. Brubaker, and Danielle Albers Szafir. 2022. Scholastic: Graphical Human-AI Collaboration for Inductive and Interpretive Text Analysis. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (Bend, OR, USA) (UIST '22)*. Association for Computing Machinery, New York, NY, USA, Article 30, 12 pages. <https://doi.org/10.1145/3526113.3545681>
 - [35] Eric Horvitz. 1999. Principles of Mixed-Initiative User Interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Pittsburgh, Pennsylvania, USA) (CHI '99)*. Association for Computing Machinery, New York, NY, USA, 159–166. <https://doi.org/10.1145/302979.303030>
 - [36] Interaction Institute for Social Change. 2018. Power Dynamics: The Hidden Element to Effective Meetings. <https://interactioninstitute.org/power-dynamics-the-hidden-element-to-effective-meetings/>.
 - [37] Daphne Ippolito, Ann Yuan, Andy Coenen, and Sehmon Burnam. 2022. Creative Writing with an AI-Powered Writing Assistant: Perspectives from Professional Writers. arXiv:2211.05030 [cs.HC]
 - [38] iResearchNet. 2016. Team Mental Model. <https://psychology.iresearchnet.com/industrial-organizational-psychology/group-dynamics/team-mental-model/>.
 - [39] Anthony Jameson, Stephan Baldes, and Thomas Kleinbauer. 2003. Enhancing mutual awareness in group recommender systems. In *Proceedings of the IJCAI*, Vol. 10.
 - [40] Jialun Aaron Jiang, Kandrea Wade, Casey Fiesler, and Jed R. Brubaker. 2021. Supporting Serendipity: Opportunities and Challenges for Human-AI Collaboration in Qualitative Analysis. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW1, Article 94 (apr 2021), 23 pages. <https://doi.org/10.1145/3449168>
 - [41] Neringa Kalpokas Jörg Hecker. 2023. *The Ultimate Guide to Qualitative Research - Part 1: The Basics*. Retrieved December 9, 2023 from <https://atlasti.com/guides/qualitative-research-guide-part-1/theoretical-perspective>
 - [42] Karen S Kurasaki. 2000. Intercode reliability for validating conclusions drawn from open-ended interview data. *Field methods* 12, 3 (2000), 179–194. <https://doi.org/10.1177/1525822X0001200301>
 - [43] Jonathan Lazar, Jinjuan Heidi Feng, and Harry Hochheiser. 2017. *Research methods in human-computer interaction*. Morgan Kaufmann.
 - [44] John D Lee and Katrina A See. 2004. Trust in automation: Designing for appropriate reliance. *Human factors* 46, 1 (2004), 50–80. <https://doi.org/10.1518/hfes.46.1.50.30392>
 - [45] Moira Maguire and Brid Delahunt. 2017. Doing a thematic analysis: A practical, step-by-step guide for learning and teaching scholars. *All Ireland Journal of Higher Education* 9, 3 (2017).
 - [46] Carmel Maher, Mark Hadfield, Maggie Hutchings, and Adam De Eyto. 2018. Ensuring rigor in qualitative data analysis: A design research approach to coding combining NVivo with traditional material methods. *International journal of qualitative methods* 17, 1 (2018), 1609406918786362.
 - [47] Thomas W. Malone and Kevin Crowston. 1994. The Interdisciplinary Study of Coordination. *ACM Comput. Surv.* 26, 1 (mar 1994), 87–119. <https://doi.org/10.1145/174666.174668>
 - [48] Mika V Mäntylä, Bram Adams, Foutse Khomh, Emelie Engström, and Kai Petersen. 2015. On rapid releases and software testing: a case study and a semi-systematic literature review. *Empirical Software Engineering* 20 (2015), 1384–1425.
 - [49] Megh Marathe and Kentaro Toyama. 2018. Semi-Automated Coding for Qualitative Research: A User-Centered Inquiry and Initial Prototypes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3173574.3173922>
 - [50] Nora McDonald, Sarita Schoenebeck, and Andrea Forte. 2019. Reliability and Inter-Rater Reliability in Qualitative Research: Norms and Guidelines for CSCW and HCI Practice. *Proc. ACM Hum.-Comput. Interact.* 3, CSCW, Article 72 (nov 2019), 23 pages. <https://doi.org/10.1145/3359174>
 - [51] Mary L. McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.
 - [52] Andrew Moravcsik. 2014. Transparency: The Revolution in Qualitative Research. *PS: Political Science; Politics* 47, 1 (2014), 48–53. <https://doi.org/10.1017/S1049096513001789>
 - [53] Michael Muller, Shion Guha, Eric P.S. Baumer, David Mimno, and N. Sadat Shami. 2016. Machine Learning and Grounded Theory Method: Convergence, Divergence, and Combination. In *Proceedings of the 2016 ACM International Conference on Supporting Group Work (Sanibel Island, Florida, USA) (GROUP '16)*. Association for Computing Machinery, New York, NY, USA, 3–8. <https://doi.org/10.1145/2957276.2957280>
 - [54] Helen Noble and Joanna Smith. 2015. Issues of validity and reliability in qualitative research. *Evidence-Based Nursing* 18, 2 (2015), 34–35. <https://doi.org/10.1136/eb-2015-102054>
 - [55] Gary M. Olson and Judith S. Olson. 2000. Distance Matters. *Hum.-Comput. Interact.* 15, 2 (sep 2000), 139–178. https://doi.org/10.1207/S15327051HCI1523_4
 - [56] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]
 - [57] Clodhna O'Connor and Helene Joffe. 2020. Intercode reliability in qualitative research: debates and practical guidelines. *International journal of qualitative methods* 19 (2020), 1609406919899220. <https://doi.org/10.1177/1609406919899220>
 - [58] Harshada Patel, Michael Pettitt, and John R. Wilson. 2012. Factors of collaborative working: A framework for a collaboration model. *Applied Ergonomics* 43, 1 (2012),

- 1–26. <https://doi.org/10.1016/j.apergo.2011.04.009>
- [59] I. J. Pérez, F. J. Cabrerizo, S. Alonso, Y. C. Dong, F. Chiclana, and E. Herrera-Viedma. 2018. On dynamic consensus processes in group decision making problems. *Information Sciences* 459 (2018), 20–35. <https://doi.org/10.1016/j.ins.2018.05.017>
- [60] K. Andrew R. Richards and Michael A. Hemphill. 2018. A practical guide to collaborative qualitative data analysis. *Journal of Teaching in Physical Education* 37, 2 (2018), 225–231. <https://doi.org/10.1123/jtpe.2017-0084>
- [61] Tim Rietz and Alexander Maedche. 2021. Cody: An AI-Based System to Semi-Automate Coding for Qualitative Research. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI '21). Association for Computing Machinery, New York, NY, USA, Article 394, 14 pages. <https://doi.org/10.1145/3411764.3445591>
- [62] Johnny Saldaña. 2021. *The coding manual for qualitative researchers*. SAGE publications Ltd. 1–440 pages.
- [63] Hannah Snyder. 2019. Literature review as a research methodology: An overview and guidelines. *Journal of Business Research* 104 (2019), 333–339. <https://doi.org/10.1016/j.jbusres.2019.07.039>
- [64] A. Teherani, T. Martimianakis, T. Stenfors-Hayes, A. Wadhwa, and L. Varpio. 2015. Choosing a Qualitative Research Approach. *J Grad Med Educ* 7, 4 (Dec 2015), 669–670. <https://doi.org/10.4300/JGME-D-15-00414.1>
- [65] Kenneth W. Thomas. 2008. Thomas-kilmann conflict mode. *TKI Profile and Interpretive Report* 1, 11 (2008).
- [66] Daphne C. Watkins. 2017. Rapid and Rigorous Qualitative Data Analysis: The “RADaR” Technique for Applied Research. *International Journal of Qualitative Methods* 16, 1 (2017), 1609406917712131. <https://doi.org/10.1177/1609406917712131> arXiv:<https://doi.org/10.1177/1609406917712131>
- [67] Ziang Xiao, Xingdi Yuan, Q. Vera Liao, Rania Abdelghani, and Pierre-Yves Oudeyer. 2023. Supporting Qualitative Analysis with Large Language Models: Combining Codebook with GPT-3 for Deductive Coding. In *Companion Proceedings of the 28th International Conference on Intelligent User Interfaces* (Sydney, NSW, Australia) (IUI '23 Companion). Association for Computing Machinery, New York, NY, USA, 75–78. <https://doi.org/10.1145/3581754.3584136>
- [68] Himanshu Zade, Margaret Drouhard, Bonnie Chinh, Lu Gan, and Cecilia Aragon. 2018. Conceptualizing Disagreement in Qualitative Coding. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3173574.3173733>

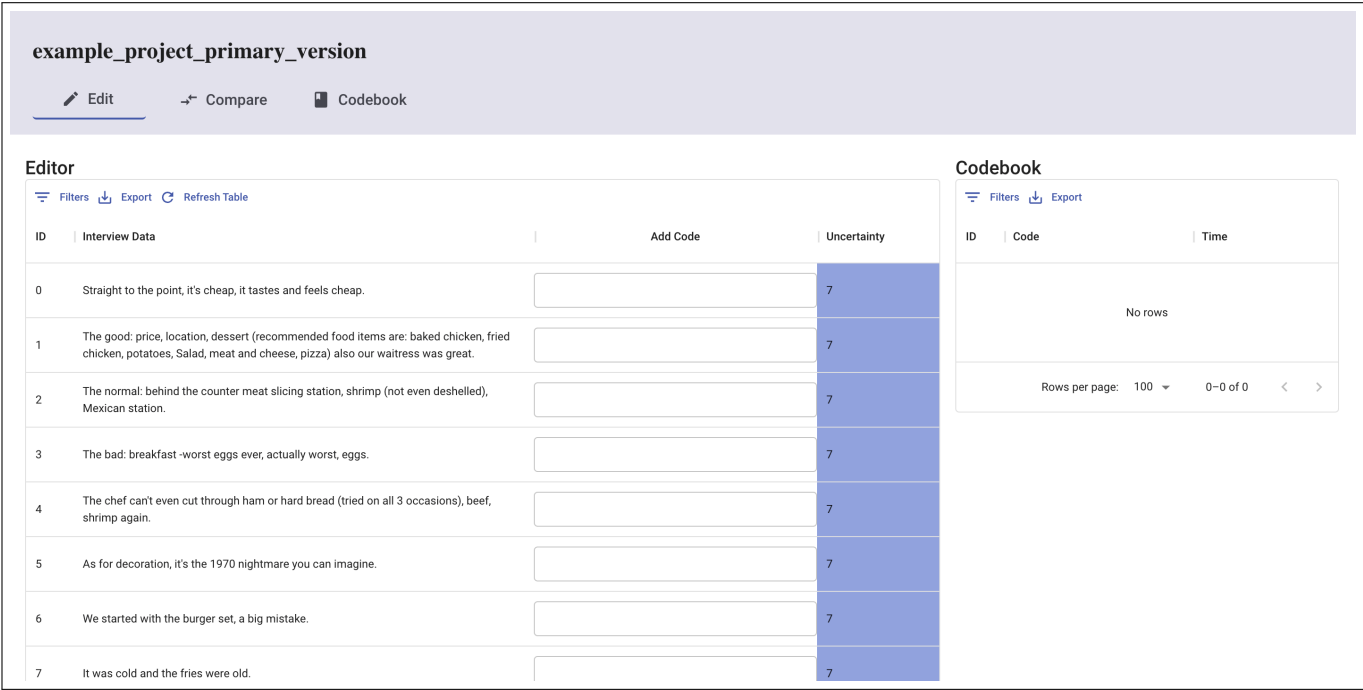


Figure 9: Primary Prototype for Phase 1.

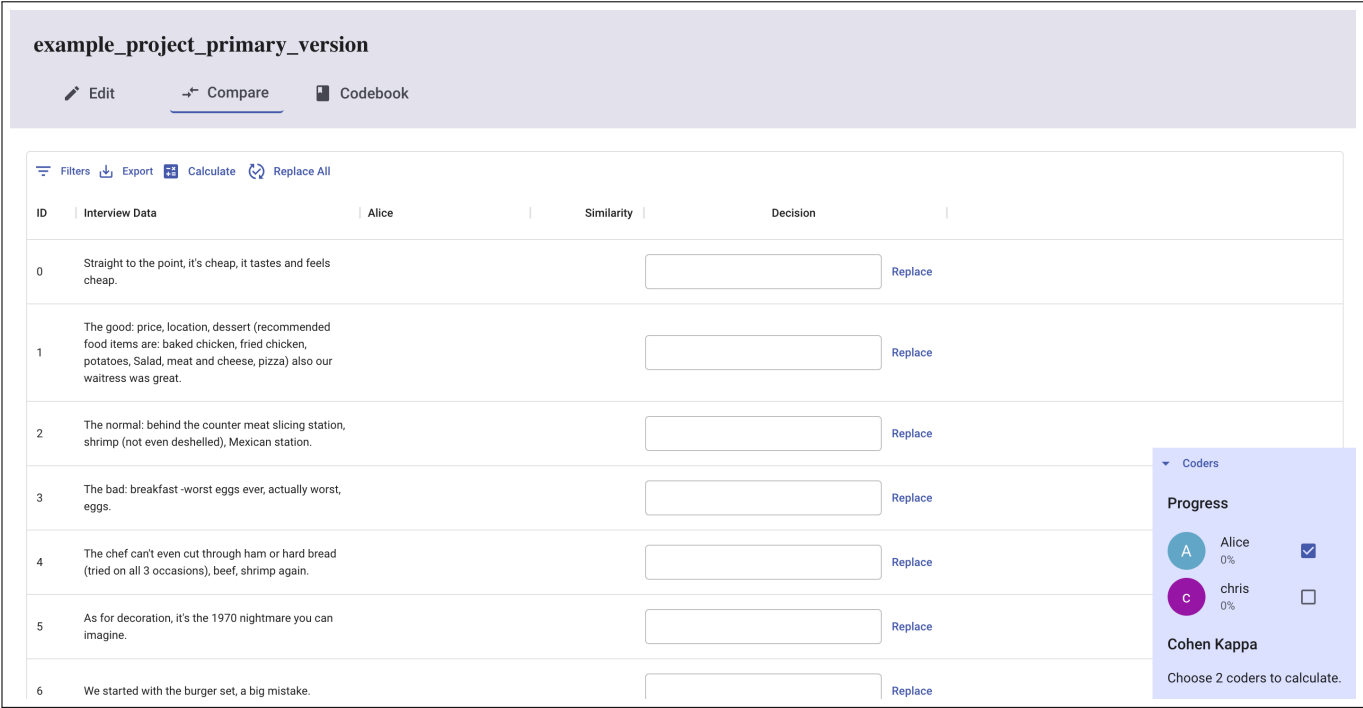


Figure 10: Primary Prototype for Phase 2.

Table 4: Different CQA Software. Note: This list is based on public online resources and not exhaustive.

Application	Atlas.ti Desktop	Atlas.ti Web	NVivo Desktop	Google docs	MaxQDA
Collaboration ways	Coding separately and then export the project bundles to other coders	Coding on the same web page	Coding separately and then export the project bundles to other coders	Collaborative simultaneously	Provide master project that includes documents and primary codes, and then send copies to others, allowing them to merge
Coding phase	All Phases	All phases	All Phases	All phases	All Phases
Independence	independent	not independent	Inpedendent	not independent	Inpedendent
Synchrony	Asynchronous	Synchronous	Asynchronous	Synchronous	Asynchronous
Unit of analysis	Select any text	Select any text	Select any text, but calculation of IRR can be on character, sentence, paragraph	Select any text	Select any text
IRR	Agreement Percentage; Holsti Index; Krippendorff's family of Alpha	NA	Agreement Percentage; Kappa coefficient	NA	Agreement Percentage; Kappa coefficient
Calculation of IRR	Calculating after coding system is stable and all codes are defined	Calculating manually at any time	Calculating after coding system is stable and all codes are defined	NA	Calculating after coding system is stable and all codes are defined
Multi-valued coding	support multiple codes	support multiple codes	support multiple codes	support multiple codes	support multiple codes
Uncertainty/ Disagreements	NA	NA	quickly identify areas of agreement and disagreement within the source data using the green, yellow, and blue indicators on the scroll bar.	NA	NA

example_project_primary_version			
Edit Compare Codebook			
Filters Export			
Code Group	Unique Code	User Code	Interview Data
Food	cheap food	cheap food	Straight to the point, it's cheap, it tastes and feels cheap.
Service	good service	good service	The good: price, location, dessert (recommended food items are: baked chicken, fried chicken, potatoes, Salad, meat and cheese, pizza) also our waitress was great.
Others	normal part	normal part	The normal: behind the counter meat slicing station, shrimp (not even deshelled), Mexican station.
Food	bad food	bad food	The bad: breakfast -worst eggs ever, actually worst, eggs.
Service	bad service	bad service	The chef can't even cut through ham or hard bread (tried on all 3 occasions), beef, shrimp again.
Service	bad environment	bad environment	As for decoration, it's the 1970 nightmare you can imagine.
Food		bad food	We started with the burger set, a big mistake.
Service	bad service	bad service	It was cold and the fries were old.
Service		bad service	Mentioned to our waitress, but nothing came of it.
Food	pricey food	pricey food	We ordered a large thin pizza which was a bit pricey after adding toppings.
Food	great pizza	great pizza	Pizza was ok, not great.
Service		good service	Aside from not being our advocate, our server was good.

Figure 11: Primary Prototype for Phase 3. The gray-colored codes serve as an example to illustrate the differences between "Code Group", "Unique Code", and "User Code". The interfaces shown above, being preliminary mockups, were utilized to gather feedback from our primary interviewees in Step3, Section 4.1, for the refinement of the final version of interfaces including Figure 5, 6, and 7.

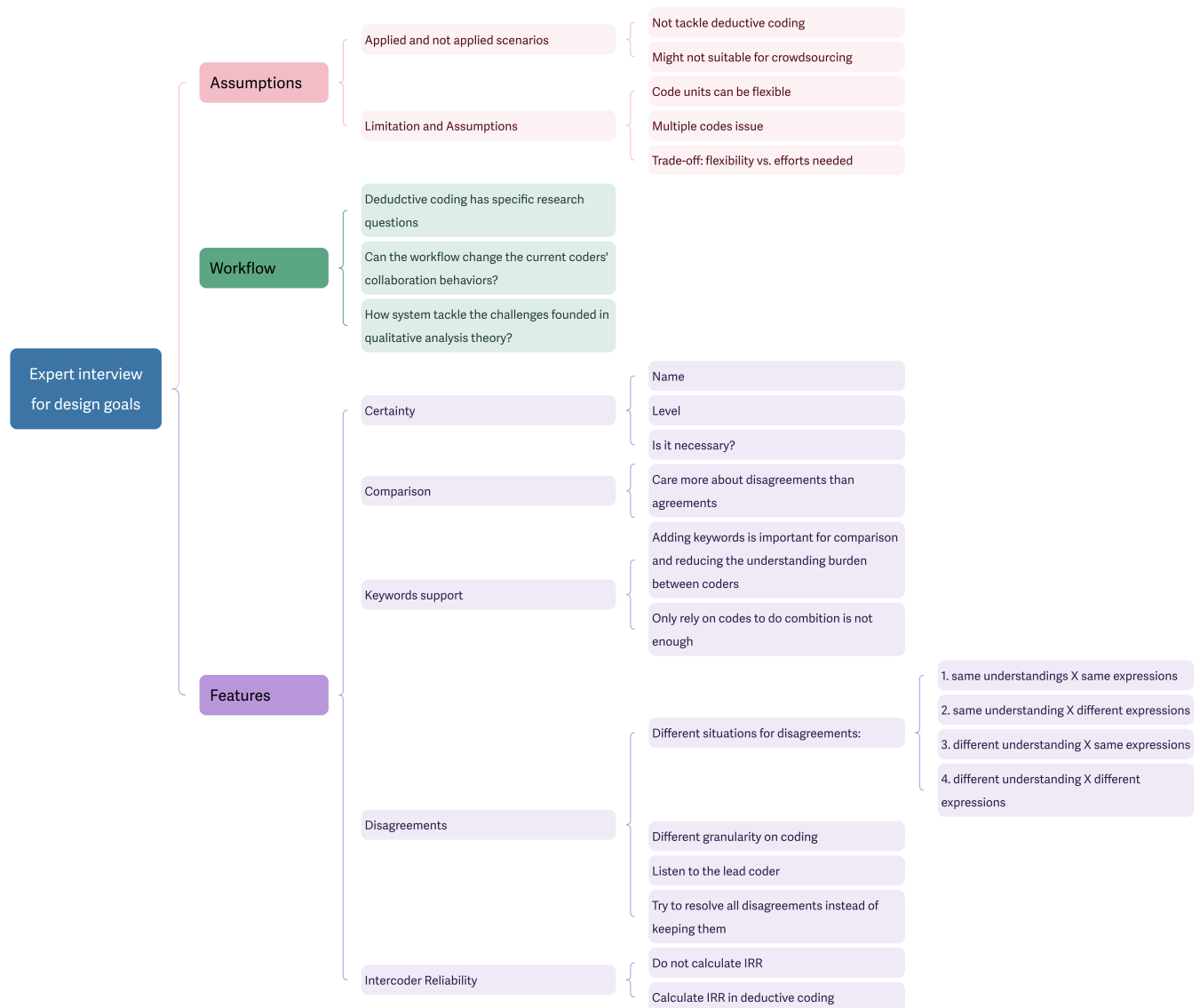


Figure 12: Results of thematic analysis in Step3 (Section 4.1) from expert interviews to derive design goals, with each node representing a coded element.

Table 5: The prompts utilized in CollabCoder in Phase 1 when communicating with the ChatGPT API to produce code suggestions for text.

Phases	Features	Prompt Template	Example
Phase 1	Seek code suggestions for units	• system role: You are a helpful qualitative analysis assistant, aiding researchers in developing codes that can be utilized in subsequent stages, including discussions for creating codebooks and final coding processes; • user input: Please create three general summaries for [text] (within six-word);	[Text]: "How A Business Works was an excellent book to read as I began my first semester as a college student. Although my goal is to major in Business, I started my semester off with no idea of even the basic guidelines a Business undergrad should know. This book describes in detail every aspect dealing with business relations, and I enjoyed reading it. It felt great going to my additional business classes prepared and knowledgeable on the subject they were describing. Very well written, Professor Haeberle! I recommend this book to anyone and everyone who would like additional knowledge pertaining to business matters." Three general summaries for the above [Text]: 1. Book enlightened my initial business journey. 2. Comprehensive guide for business undergraduates. 3. Knowledge boost for new business students.
	Seek most relevant codes from coding history	• system role: You are a helpful qualitative analysis assistant, aiding researchers in developing codes that can be utilized in subsequent stages, including discussions for creating codebooks and final coding processes; • user input: Please identify the top three codes relevant to this [text] from the following [Code list]; 1. [Code] 2. [Code] ... Here is the example format of results: 1. code content 2. code content 3. code content	[Text] "How A Business Works was an excellent book to read as I began my first semester as a college student. Although my goal is to major in Business, I started my semester off with no idea of even the basic guidelines a Business undergrad should know. This book describes in detail every aspect dealing with business relations, and I enjoyed reading it. It felt great going to my additional business classes prepared and knowledgeable on the subject they were describing. Very well written, Professor Haeberle! I recommend this book to anyone and everyone who would like additional knowledge pertaining to business matters." [Code list] 1. Detailed introduction to business relations 2. Inspiring guide to improve life 3. Journey of light and love. 4. Easy to read, highlight-worthy 5. Well-written lesson on simplicity 6. Rodriguez tells truth, Pelosi lies Three relevant codes to [Text] from [Code list]: 1. Detailed introduction to business relations 2. Easy to read, highlight-worthy. 3. Well-written lesson on simplicity.

Table 6: The prompts utilized in CollabCoder in Phase 2 when communicating with the ChatGPT API to produce code suggestions for text.

Phases	Features	Prompt Template	Example
Phase 2	Make code decisions	• system role: You are a helpful qualitative analysis assistant, aiding researchers in developing final codes that can be utilized in subsequent stages, including final coding processes; • user input: Please create three concise, non-repetitive, and general six-word code combinations for the [text] using [Code1] and [Code2]: 1. [Code] 2. [Code] ... Requirements: 1. 6 words or fewer; 2. No duplicate words; 3. Be general; 4. Three distinct versions Here is the format of results: Version1: code content Version2: code content Version3: code content	[Text] "How A Business Works was an excellent book to read as I began my first semester as a college student. Although my goal is to major in Business, I started my semester off with no idea of even the basic guidelines a Business undergrad should know. This book describes in detail every aspect dealing with business relations, and I enjoyed reading it. It felt great going to my additional business classes prepared and knowledgeable on the subject they were describing. Very well written, Professor Haeberle! I recommend this book to anyone and everyone who would like additional knowledge pertaining to business matters." [Code1]: Detailed introduction to business relations. [Code2]: Comprehensive guide to business basics Three suggestions for final codes: Version1: In-depth overview of business fundamentals Version2: Thorough guide to business relationships Version3: Comprehensive resource on business essentials

Table 7: The prompts utilized in CollabCoder in Phase 3 when communicating with the ChatGPT API to produce code group suggestions for final code decisions.

Phases	Features	Prompt Template	Example
Phase 3	Generate code groups	• system role: You are a helpful qualitative analysis assistant, aiding researchers in generating final code groups/main themes based on the [Code list] provided, in order to give an overview of the main content of the coding. • user input: Organize the following [Code list] into 3 thematic groups without altering the original codes, and name each group: 1. [Code] 2. [Code] ... Here is the format of the results: Group1: [theme] 1.[code] 2.[code] 3.[code]	[Code list]: 1. Simplified business knowledge 2. Unconventional, but valuable business insights. 3. Effective lessons on simplicity 4. Innovative leadership through Jugaad. 5. Cautionary book on costly Google campaigns. 6. Timeless love principles improve business. 7. Politicians deceive for political gain. 8. A high school must-read for financial literacy. 9. Entertaining and educational graphic novel. Three code groups for the above [Code list]: Group1: Simplified business knowledge 1. Simplified business knowledge 2. Effective lessons on simplicity 3. Cautionary book on costly Google campaigns. Group2: Inspiring and practical personal development book 1. Timeless love principles improve business. 2. A high school must-read for financial literacy. 3. Entertaining and educational graphic novel. Group3: Unconventional, but valuable business insights 1. Innovative leadership through Jugaad. 2. Unconventional, but valuable business insights. 3. Politicians deceive for political gain.

Table 8: Demographics of Participants in User Evaluation. Note: QA expertise is not solely determined by the number of QA experiences, but also by the level of QA knowledge. This is why some participants with 1-3 times of prior experience may still regard themselves as having intermediate expertise.

Pairs		English	Job	Education	Related experience	Self-reported QA expertise	QA times	Software for QA
Pair 1	P1	Proficient	Student	Master	Basic understanding of qualitative research method	No Experience	None	None
	P2	First language	Automation QA Engineer	Undergraduate	Automation	No Experience	None	None
Pair 2	P3	First language	Phd Student	PhD and above	HCI	Expert	7 times above	Atlas.ti Desktop
	P4	First language	Undergraduate	Undergraduate	Business analytics with Python and R	No Experience	None	None
Pair 3	P5	Proficient	Student	Undergraduate	Coding with Python	Beginner	1-3 times	None
	P6	First language	Research Assistant	Master	Asian studies	Expert	7 times above (mainly interview data)	Word, Excel, Dedoose
Pair 4	P7	First language	Data Analyst	Undergraduate	Data Visualisation	No Experience	None	None
	P8	First language	Student	Undergraduate	R, HTML/CSS, Market research	Beginner	1-3 times	R
Pair 5	P9	First language	Research assistant	Undergraduate	Learning science, Grounded theory	Intermediate	4-6 times	NVivo
	P10	First language	Data science intern	Undergraduate	Computer Vision, Python	No Experience	None	None
Pair 6	P11	First language	Behavioral Scientist	Undergraduate	Psychology, Behavioral Science, Thematic analysis	Intermediate	1-3 times	Word
	P12	First language	Student	Undergraduate	Accounting & Python, SQL	No experience	None	None
Pair 7	P13	First language	Research Assistant	Undergraduate	SPSS, Python, basic qualitative analysis understanding, topic modeling	Beginner	1-3 times	None
	P14	First language	Research Assistant	Undergraduate	Have research experience using QA for interview transcription	Intermediate	7 times above	NVivo, Excel
Pair 8	P15	First language	Researcher	Master	Thematic analysis for interview, literature review	Beginner	1-3 times	fQCA
	P16	First language	Student	Master	Social science	No experience	None	None

Table 9: Observation notes for Pair1-Pair4. The language has been revised for readability.

	Atlas.ti Web	CollabCoder
P1xP2	Even individuals familiar with Google Docs/Excel might find it challenging to adapt to Atlas.ti. P1's learning pace was even slower than P2's. P1's coding was more detailed and extensive than P2's, making his codes longer and more content-rich. His lack of experience in ML further hampered his speed, causing a significant delay in the coding process. In a 30-minute span, he managed only 5 codes compared to another coder who completed all 20. Due to time constraints, we had to stop the process. Over time, the involvement of the other coder diminished. Only one coder, more adept with the platform, primarily handled the coding process. The other coder, like P1, shifted to a supportive role, offering input on the final report and the categorization phase.	This time around, P1 found it easier to start coding. Both he and the other coder seldom used the "Similar codes" function. Additionally, they rarely used the "certainty" button, indicating a potential issue of over-reliance on certain features or methods.
P3xP4	Even the expert coder (P3) faced challenges learning the software and initially felt lost navigating its interface. Additionally, she found it difficult to identify the origin of selected text when only a portion is chosen from the original unit. In both coding sessions, the lead coder shares her screen and invites the other coder to offer suggestions for combining codes and arriving at the final code. Due to the limitations of the software, they are obliged to manually search for similar codes, relying on visual inspection to group them together.	P3 is a conscientious coder who is concerned about potentially slowing down the overall pace of the study. To address this, she intermittently checks the progress of others to adjust her own workflow. She finds this feature to be "quite helpful." When it comes to coding, if the codes are identical, they typically don't consult definitions. While ChatGPT serves as a reference point for decision-making, it is not strictly followed. If there's a difference in understanding, they will refer to ChatGPT for final decision support. When P4 sets the certainty level to 2, it signifies "I'm not sure what this person is talking about." The lead coder is conscious of not overly relying on her own codes, as she doesn't want to appear too dominant within the team. By using third-party codes, she aims to maintain a more balanced influence. P4 also mentioned that he sometimes assigns low priority to definitions because he has only a few to refer to. During the decision-making process, direct selections from ChatGPT suggestions have become less frequent. Instead, the team tends to use ChatGPT is more of a point of reference. This seems to indicate that they are becoming more cautious in their approach.
P5xP6	Beginners have the option of referring to others' codes as a starting point for their own coding endeavors. P6, for instance, prefers to check the code history. This approach can provide valuable insights and context, helping new coders understand the coding process better and potentially speeding up their learning curve. The team takes advantage of the auto-completion function, typing in just a few words and then clicking the check button to select existing codes instead of creating new ones. When P5 is coding, he initially refers to other people's codes before adding his own. The codes generated by both coders tend to be rather general. They often refer to each other's work, with the beginner usually following the coding scheme established by the more experienced coder.	Russell is not familiar with the new coding method and initially selected the entire text as "keywords support", thinking that only the selected portion would be coded. This suggests that users may need some training to effectively use the coding system.
P7xP8	To speed up the coding process, only one coder takes on the responsibility of combining and grouping the codes into thematic clusters.	P7 and P8 both tend to use ChatGPT sparingly, favoring the creation of their own codes. P7 mentions that the long latency for ChatGPT's suggestions is a factor; if the results aren't quick, he opts to input his own codes. P8 notes that she often has to edit ChatGPT's suggestions, deleting some words to better tailor them to her needs. However, they are more likely to choose suggestions from ChatGPT if they want to expedite the process. Even if they don't ultimately select a ChatGPT suggestion, they still refer to these codes as a reference point. This approach aligns with the sentiment that AI can't be trusted for every task; it serves as a tool rather than a definitive authority. If there's any uncertainty about why a code is part of a specific group, or if the meaning of a code within a group is unclear, they will refer back to the original text during the code grouping phase for clarification. By highlighting keywords and listing them, the coders are able to work asynchronously instead of in real-time. This approach allows each coder to leave markers of their understanding, facilitating a smoother integration of perspectives without the need for immediate discussion.

Table 10: Observation notes for Pair5-Pair8. The language has been revised for readability.

	Atlas.ti Web	CollabCoder
P9xP10	Normal collaboration process, no specific notes	The coding process involves multiple steps: initially reading the data, requesting suggestions, reviewing those suggestions, returning to the raw data for another check, and then choosing or editing the code. After this, keywords are added and the level of certainty is labeled. When it comes to merging codes, the team starts with a preliminary idea for a final decision, and then consults ChatGPT to generate a final, merged code.
P11xP12	P12 adopts a strategy of starting his coding from the last data point and working his way to the top, in an effort to minimize overlap and influence from P11. The pace at which each coder works varies significantly: one coder is much faster than the other and, consequently, contributes more to the overall workload. In time-sensitive situations, the quicker coder naturally takes on more responsibilities than the slower coder. This dynamic could have implications for the diversity and depth of coding, as the faster coder's perspectives may disproportionately influence the final output.	A less-than-ideal scenario for discussion. The team may overly rely on ChatGPT's generated decisions due to time constraints. In these cases, substantive discussion is replaced with shortcuts like simply choosing "the first one" or "the second one" from ChatGPT's suggestions. This is a notable drawback for the research, as it sidesteps deeper analysis and thought, leading to concerns about over-reliance on automated suggestions. The dynamic often results in the more experienced coder taking a dominant role in the process, which could impact the diversity of perspectives in the coding. AI does offer the advantage of allowing users to work with longer text segments compared to manual coding, which often focuses on keywords or smaller data units. However, this advantage should not come at the expense of thoughtful discussion and careful consideration in the coding process.
P13xP14	Normal collaboration process, no specific notes	The overall coding process appears to be smooth. Both coders generally agree and neither is overly dominant; they respect each other's opinions. Because of this harmony, they utilize ChatGPT to generate the final code expressions. It seems that AI plays a significant role in the code grouping process, directing the way codes are organized. When the coders are working independently, they tend to group codes based on sentiment analysis. However, under AI's guidance, their focus shifts to content analysis. This suggests that while AI can be a helpful tool, its influence can also steer the analytical direction, which may or may not align with the coders' initial approach or intentions.
P15xP16	Due to time constraints, discussions between the coders are notably brief and to the point. There isn't much room for extended dialogue or deeper analysis, which could have implications for the thoroughness and quality of the coding process.	Participants generally start by reading the original text, then request suggestions from ChatGPT before proceeding to code the data. P16 follows this sequence, reading the text first and then consulting ChatGPT's suggestions. P15, on the other hand, sometimes deletes her initial code entry to generate a different version. Due to time constraints, she doesn't delve deeply into the text and may even skip over some sections. To save time, she might initiate ChatGPT's code suggestion process for another text segment while working on the current one. Both P15 and P16 demonstrate mutual respect when their codes closely align (with a similarity score greater than 0.9). They don't particularly mind whose code is used for the final decision. For instance, they may choose one of P15's codes for one text segment and switch to another code from P16 for a different segment. If their coding doesn't match despite having similar evidence, they discuss the reasons behind their code choices. The coder with the more explainable rationale usually wins out, with the other coder simply saying, "Use yours." If they can't reach a decision, they turn to ChatGPT for a suggested code. When neither coder feels confident in their understanding of the raw data, they'll admit their uncertainty, often stating, "I have no idea about this", before potentially seeking further guidance.